

Secure Patient Data & Insurance Management System

A Project Report

Submitted in partial fulfillment of the
Requirements for the award of the Degree of
BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)

By

Mohd Aadil

24BIT054/45

**Under the esteemed guidance of
Mr. Wilson Rao and Ms. Bertilla Fernandes**



**DEPARTMENT
OF
INFORMATION TECHNOLOGY
JAI HIND COLLEGE (Empowered Autonomous)
MUMBAI, 400020
MAHARASHTRA
2026-27**

JAI HIND COLLEGE

(Empowered Autonomous)

DEPARTMENT OF INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project entitled, “**Secure Patient Data & Insurance Management System**”, is bonafied work of **Mohd Aadil** bearing UID / Roll No.: **24BIT054/45** submitted in partial fulfilment of the requirements for the award of degree of BACHELOR OF SCIENCE in INFORMATION TECHNOLOGY from Jai Hind College Empowered Autonomous (University of Mumbai).

Internal Guide

Coordinator

External Examiner

Date:

College Seal

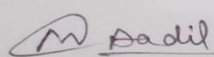
DECLARATION

I hereby declare that the project entitled, “**Secure Patient Data & Insurance Management System**” done at Jai Hind College (Empowered Autonomous), has not been in any case duplicated to submit to any other university for the award of any degree. To the best of my knowledge other than me, no one has submitted to any other university.

The project is done in partial fulfillment of the requirements for the award of degree of BACHELOR OF SCIENCE (INFORMATIONTECHNOLOGY) to be submitted as Semester V project as part of our curriculum.

Name and Signature of the Student

Mohd Aadil

A handwritten signature in dark ink, appearing to read 'M Aadil', is written on a light-colored rectangular background.

ACKNOWLEDGEMENT

I am extremely grateful for the guidance of our Head of Department (Information Technology & Software Development) **Mr. Wilson Rao**. Sir had great involvement in making sure my project is a well-rounded and a flawless system by constantly guiding us till the completion of our project work by providing all the necessary information for developing a good system.

I would like to express immense gratitude to the people who have helped me throughout the course of my project. I am grateful to **Prof. Ms. Bertilla Fernandes** for her constant encouragement and support.

I would also like to thank all of my friends and my seniors who supported and helped me in completing the project, where they all had their own interesting takes on the technology stack, and the own interesting ideas on how to finesse the system even further. I would also like to thank my family for their constant support and encouragement.

TABLE OF CONTENTS

1. INTRODUCTION	8
1.1 Background	8
1.2 Objectives	9
1.3 Purpose, Scope, and Applicability	10
1.3.1 Purpose	10
1.3.2 Scope	10
1.3.3 Applicability	10
1.4 Achievements	11
1.5 Organization of Report	12
2. SURVEY OF TECHNOLOGIES	13
3. REQUIREMENT AND ANALYSIS	14
3.1 Problem Definition	14
3.2 Requirement Specification	15
3.3 Planning and Scheduling	16
3.4 Software and Hardware Requirements	19
3.5 Preliminary Product Description	20
3.6 Conceptual Models	21
4. SYSTEM DESIGN	36
4.1 Basic Modules	36
4.2 Data Design	37
4.2.1 Schema Design	37
4.2.2 Data Integrity and Constraints	38
4.3 Procedural Design	39
4.3.1 Logic Diagrams	39
4.3.2 Data Structures	39
4.3.3 Algorithms Design	40
4.4 User interface design	41
4.5 Security Issues	43
4.6 Test Cases Design	44

5. IMPLEMENTATION AND TESTING	45
5.1 Implementation Approach	45
5.2 Coding Details and Code Efficiency	46
5.2.1 Code Efficiency	49
5.3 Testing Approach	62
5.3.1 Unit Testing	62
5.3.2 Integration Testing	63
5.3.3 System Testing	64
5.4 Modifications and Improvements	66
5.5 Test Cases	67
 6. RESULTS AND DISCUSSION	 69
6.1 Test Reports	69
6.2 User Documentation	71
 7. CONCLUSION	 78
7.1 Conclusion	78
7.2 Limitations of the System	79
7.3 Future Scope of the Project	80
 REFERENCES.....	 81
GLOSSARY	82

List of Figures

1. Gantt Chart	17
2. PERT Chart	18
3. Event Table	22
4. ER Diagram	23
5. Class Diagram	24
6. Object Diagram	25
7. Use Case Diagram	26
8. Activity Diagram	27
9. Sequence Diagram	28
10. State Diagram	29
11. Package Diagram	30
12. Component Diagram	31
13. Deployment Diagram	32
14. Data Flow Level 0 Diagram	33
15. Data Flow Level 1 Diagram	34
16. Data Flow Level 2 Diagram	35
17. Database Schema Design	37

CHAPTER 1: INTRODUCTION

1.1 Background

Hospitals and healthcare organizations handle a large amount of patient information every day, including personal details, medical records, prescriptions, diagnosis reports, treatment information, bills, and insurance claims. Managing all this information manually can be time-consuming and may result in problems such as duplicate records, data entry errors, difficulty in finding information, and delays in processing patient services.

Insurance claim processing is another important part of healthcare management. Patients need to submit medical and billing information when applying for a claim, while insurance companies need to verify these details before approving or rejecting the claim. When this process is handled manually, checking documents and maintaining communication between hospitals and insurance companies can become difficult.

The Secure Patient Data & Insurance Management System is a web-based system developed to manage patient information, medical records, billing, and insurance claims through a centralized platform. The system provides separate access to different users such as patients, doctors, hospital staff, and insurance companies based on their roles.

Patients can view their personal and medical information, bills, and insurance claims. Doctors can manage patient medical records and treatment details, while hospital staff can manage patient and billing information. Insurance companies can review submitted claims and update their status as Pending, Under Review, Approved, or Rejected.

The system is developed using Python and Django, with PostgreSQL used for database management. By bringing the major healthcare and insurance activities into one system, the project reduces manual work, improves record accessibility, and makes the overall process more organized and efficient.

1.2 Objectives

The main objectives of the project are:

- To manage patient information in a centralized system.
- To maintain medical records, diagnosis details, and prescriptions digitally.
- To manage hospital bills and generate digital bills.
- To simplify the insurance claim submission and verification process.
- To provide secure role-based access to patients, doctors, hospital staff, and insurance companies.
- To reduce paperwork and manual errors.
- To make patient and medical records easier to search and retrieve.
- To maintain supporting medical and insurance documents digitally.
- To track insurance claims through Pending, Under Review, Approved, and Rejected stages.
- To improve communication and coordination between hospitals, patients, and insurance companies.
- To provide dashboards and reports for better monitoring of system activities.
- To improve the overall efficiency and organization of healthcare management.

1.3 Purpose, Scope, and Applicability

1.3.1 Purpose

The main purpose of this project is to make patient data, medical records, billing, and insurance activities easier to manage through a single web-based system. It reduces dependence on paperwork and helps users access required information quickly. The system also aims to improve communication between patients, doctors, hospitals, and insurance companies while keeping access to sensitive information controlled.

1.3.2 Scope

The system covers patient management, doctor and medical record management, hospital billing, insurance claims, user authentication, role-based access, dashboards, notifications, and document management. It allows users to add, view, update, search, and manage relevant records according to their permissions. The system also supports digital bill generation, medical document uploads, claim status tracking, and insurance claim verification.

The future scope of the system can include mobile application support, email or SMS notifications, cloud storage, advanced reporting, data analytics, and integration with other healthcare services.

1.3.3 Applicability

The system can be used by hospitals, clinics, doctors, billing staff, insurance companies, and patients for managing healthcare-related information and services. Hospitals can use it to maintain patient and billing records, doctors can manage medical information, and insurance companies can review and process claims. Patients can use the system to access their records, bills, and claim information.

The system is particularly useful for organizations that want to move from manual record handling to a more organized digital process and improve the efficiency of their daily healthcare and insurance operations.

1.4 Achievements

The project successfully provides the following features:

- Patient registration and profile management.
- Digital management of patient personal information.
- Medical record and prescription management.
- Doctor-wise management of patient medical information.
- Hospital bill creation, editing, and viewing.
- Automatic calculation of total bill amounts.
- Digital bill generation in PDF format.
- Insurance claim submission and verification.
- Claim approval and rejection management.
- Tracking claims through Pending, Under Review, Approved, and Rejected statuses.
- Separate access for different types of users.
- Insurance company-specific claim records.
- Dashboards for monitoring claim status and activities.
- Search and filtering of patient and medical records.
- Upload and management of medical and insurance documents.
- Form validation to reduce incorrect data entry.
- Secure authentication and role-based access control.
- Organized storage of healthcare and insurance information.
- Reduction of paperwork and manual processing.
- Improved coordination between patients, doctors, hospitals, and insurance companies.

1.5 Organization of Report

The report is divided into chapters that explain the project step by step.

Chapter 1 – Introduction: Provides the background, objectives, purpose, scope, applicability, and achievements.

Chapter 2 – Survey of Technologies: Explains the technologies and tools used in the project.

Chapter 3 – Requirements and Analysis: Describes the system requirements and analysis.

Chapter 4 – System Design: Covers database design, system architecture, and diagrams.

Chapter 5 – Implementation: Explains the development and implementation of the project modules.

Chapter 6 – Testing: Describes the testing process and test results.

Chapter 7 – Results and Discussion: Presents the results of the implemented system.

Chapter 8 – Conclusion and Future Scope: Summarizes the project and discusses possible future improvements.

CHAPTER 2: SURVEY OF TECHNOLOGIES

This chapter describes the main technologies and tools used to develop the Secure Patient Data & Insurance Management System. The selected technologies help in building a secure, reliable, and easy-to-manage web application.

2.1 Frontend Technology

HTML, CSS, and Bootstrap are used to create the user interface. HTML provides the structure of web pages, CSS is used for styling, and Bootstrap helps create responsive and user-friendly pages.

2.2 Backend Technology

Python with Django is used for the backend development. Django provides features for handling users, database operations, forms, authentication, and application logic.

Django follows the Model-View-Template (MVT) architectural pattern, which separates the data layer, business logic, and presentation layer of the application. This separation makes it easier to maintain and extend individual parts of the system, such as updating the user interface without affecting the underlying database logic, which was particularly useful while adding new features during the development of this project.

2.3 Database Technology

PostgreSQL is used as the database for storing patient details, medical records, billing information, insurance claims, and user-related data.

2.4 Django REST Framework

Django REST Framework (DRF) is used to create APIs where required. It helps the system exchange data between different parts of the application in a structured way.

2.5 Development Tools

Visual Studio Code is used for writing and managing the project code. pgAdmin is used to manage and view the PostgreSQL database.

2.6 PDF Generation

PDF generation is used for creating downloadable hospital bills and other required documents. This allows patients to keep their billing information in digital form.

2.7 Version Control

Git and GitHub are used to store and manage the project source code. They also provide a backup of the project and help track changes during development.

CHAPTER 3: REQUIREMENT AND ANALYSIS

3.1 Problem Definition

Managing patient records, medical information, hospital bills, and insurance claims manually can be difficult and time-consuming. Healthcare organizations often have to handle a large amount of information, which may result in errors, duplicate records, missing documents, and delays in finding important information.

Insurance claim processing can also become complicated when patient details, medical records, bills, and supporting documents are maintained separately. Insurance companies need to verify these details before approving or rejecting a claim, which can make the process slower when done manually.

Another issue is controlling access to sensitive patient information. Different users such as patients, doctors, hospital staff, and insurance companies should not have access to the same information. A proper system is therefore required to provide access according to the user's role.

The proposed Secure Patient Data & Insurance Management System provides a single platform where patient, medical, billing, and insurance information can be managed in an organized and secure manner. The system allows users to perform their required activities through separate modules and dashboards.

The system aims to:

- Reduce manual paperwork and data entry.
- Maintain patient and medical records digitally.
- Make billing and bill generation easier.
- Simplify insurance claim submission and verification.
- Track claims through Pending, Under Review, Approved, and Rejected stages.
- Provide role-based access to different users.
- Improve the organization and accessibility of healthcare information.
- Reduce errors and delays in healthcare and insurance-related processes.

3.2 Requirement Specification

The Secure Patient Data & Insurance Management System requires both functional and non-functional requirements to operate effectively.

Functional Requirements

- **Patient Management:** The system should allow authorized users to register, view, update, search, and manage patient information.
- **Medical Record Management:** Doctors should be able to maintain diagnosis, treatment, prescription, and medical report details.
- **Billing Management:** Hospital staff should be able to create, edit, view, and manage patient bills.
- **PDF Bill Generation:** The system should generate digital bills in PDF format for record keeping.
- **Insurance Claim Management:** Patients should be able to submit insurance claims with required policy, bill, medical, and supporting document details.
- **Claim Verification:** Insurance companies should be able to view and verify submitted claims.
- **Claim Status Management:** Claims should be managed through Pending, Under Review, Approved, and Rejected stages.
- **Authentication and Authorization:** Users should log in securely and access features according to their assigned role.
- **Dashboard and Reports:** The system should display useful information such as claim counts, recent claims, and other activities.
- **Document Management:** The system should allow authorized users to upload, store, and access relevant medical and insurance documents.

Non-Functional Requirements

- **Security:** Patient and insurance information should be protected from unauthorized access.
- **Usability:** The interface should be simple and easy to understand.
- **Performance:** Records should be retrieved and processed efficiently.
- **Reliability:** The system should maintain accurate and consistent data.

3.3 Planning and Scheduling

The project is planned in different stages so that each part of the system can be developed and tested in a systematic manner.

1. **Requirement Collection and Analysis:**
Identify the problems, users, required features, and functional requirements of the system.
2. **System and Database Design:**
Design the system architecture, database schema, ER diagram, class diagram, and other required diagrams.
3. **Module Development:**
Develop the main modules such as Patient Management, Medical Records, Billing, and Insurance Claims.
4. **Authentication and Integration:**
Implement user authentication, role-based access, and integrate all modules with the PostgreSQL database.
5. **Testing and Error Correction:**
Test individual features and complete workflows, identify errors, and make necessary corrections.
6. **Finalization and Documentation:**
Complete the user interface, reports, documentation, diagrams, screenshots, and prepare the final project for submission.

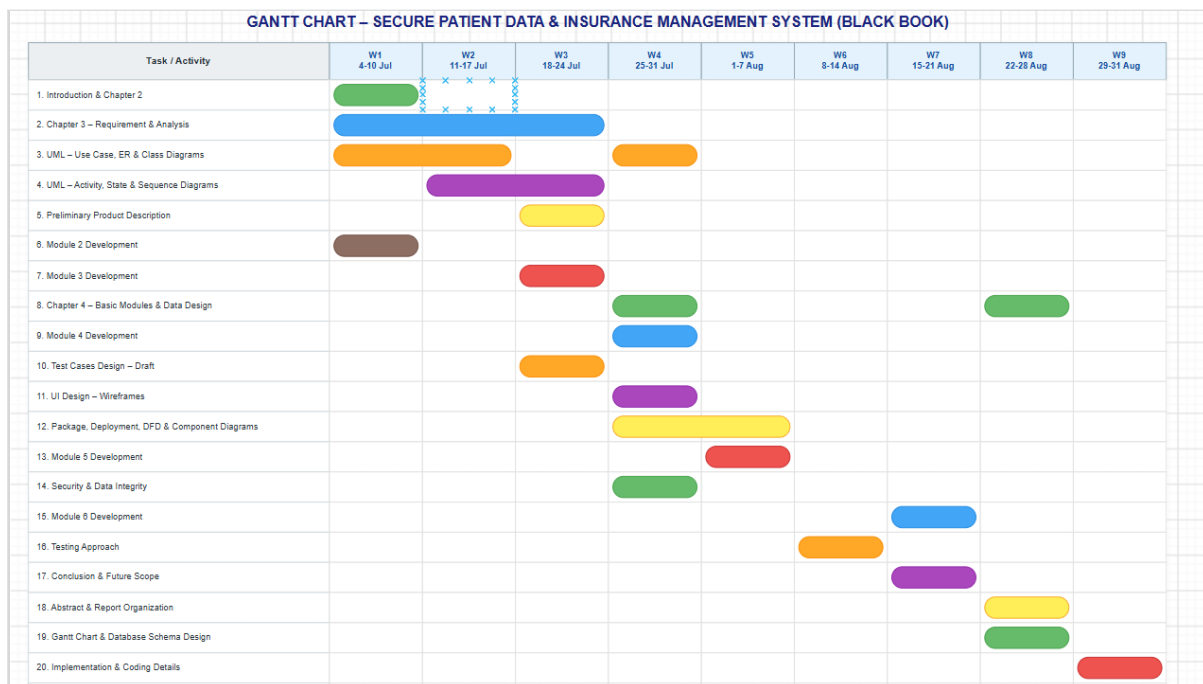


Figure 3.1: Gantt Chart

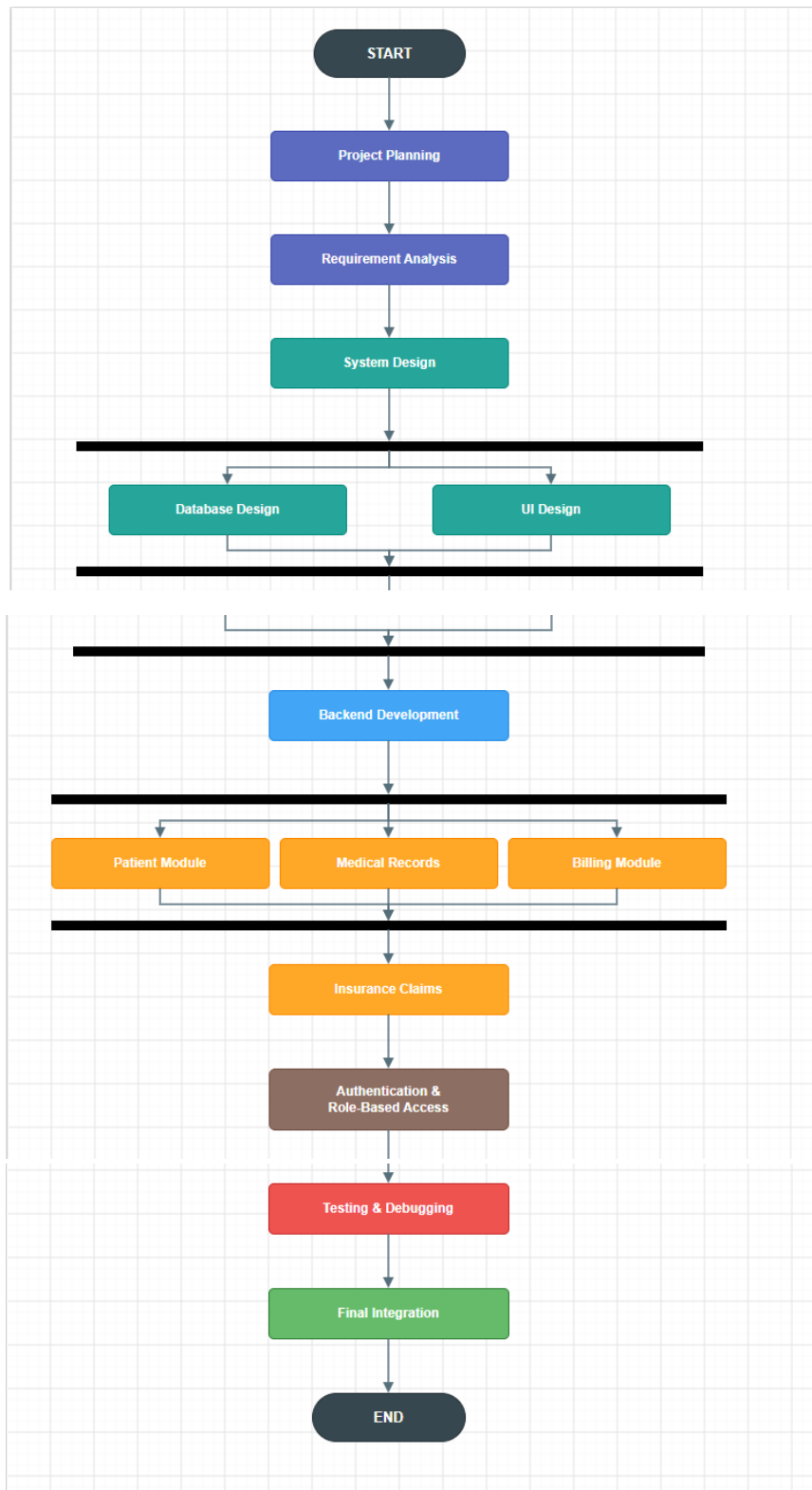


Figure 3.2: PERT Chart

3.4 Software and Hardware Requirements

Software Requirements

- Operating System: Windows
- Programming Language: Python
- Backend Framework: Django
- Database: PostgreSQL
- Frontend: HTML, CSS, Bootstrap, JavaScript
- Code Editor: Visual Studio Code
- Database Tool: pgAdmin
- Version Control: Git and GitHub

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 4 GB
- Storage: Minimum 10 GB free space
- Internet: Required for development and online services

3.5 Preliminary Product Description

The Secure Patient Data & Insurance Management System is a web-based application developed to manage important healthcare activities through a single platform. The system helps reduce manual paperwork and makes patient, medical, billing, and insurance information easier to manage.

Patients can view and manage their personal information, medical records, bills, and insurance claims. Doctors can access patient details and maintain medical records, including diagnosis, treatment, prescriptions, and medical reports. Hospital staff can register patients, manage patient information, and handle billing activities.

Insurance companies can view claims related to their organization and check the required patient, medical, and billing information before processing a claim. Claims can be tracked through different stages such as Pending, Under Review, Approved, and Rejected.

The system also provides features such as PDF bill generation, document uploading, dashboards, search and filtering, and role-based access control. Different users are provided with separate access according to their responsibilities.

The application uses Django and Python for the backend and PostgreSQL for storing system data. The overall product is designed to provide a simple, organized, and secure way to manage healthcare and insurance-related activities while reducing errors and improving efficiency.

3.6 Conceptual Models

Conceptual models are used to represent the structure and working of the Secure Patient Data & Insurance Management System before actual implementation. They help in understanding how users, data, activities, and system components are connected.

The main conceptual models used in the project are:

- **Use Case Diagram:** Shows the interaction between patients, doctors, hospital staff, insurance companies, and the system.
- **ER Diagram:** Represents the main database entities and their relationships.
- **Class Diagram:** Shows the important classes, their attributes, methods, and relationships.
- **Object Diagram:** Represents examples of objects created from the system classes and their relationships.
- **Activity Diagram:** Shows the step-by-step flow of important system processes.
- **Sequence Diagram:** Shows the interaction between users and system components in a particular process over time.
- **State Diagram:** Represents different states of system objects, such as the stages of an insurance claim.
- **Component Diagram:** Shows the major software components and how they communicate with each other.
- **Package Diagram:** Represents the organization of related classes and modules into different packages.
- **Deployment Diagram:** Shows how the application, database, server, and other system components are deployed and connected.

Secure Patient Data & Insurance Management System – Event Table				
Event	Source / Actor	Input	Process	Output
Patient Registration	Hospital Staff	Patient details	Validate and save patient information	Patient profile and Patient ID
Patient Login	Patient	Username, Password	Authenticate user	Patient Dashboard
Doctor Login	Doctor	Username, Password	Verify credentials and role	Doctor Dashboard
Add Medical Record	Doctor	Diagnosis, treatment, prescription, reports	Validate and store medical record	Updated Medical Record
Create Bill	Billing Staff	Patient, medical record, charges	Calculate total and save bill	Generated Hospital Bill
Download Bill	Patient	Bill ID	Retrieve bill and generate PDF	Downloadable PDF Bill
Submit Insurance Claim	Patient	Medical record, bill, policy details, claim amount	Validate and create claim	Claim with Pending status
Review Claim	Insurance Company	Claim ID	Check patient, medical and billing details	Claim under review
Approve Claim	Insurance Company	Claim details, approved amount	Verify and approve claim	Approved Claim
Reject Claim	Insurance Company	Claim details, remarks	Verify and reject claim	Rejected Claim
Upload Supporting Document	Patient	Medical/Insurance document	Validate and store document	Uploaded Document
View Claim Status	Patient	Claim ID	Retrieve current claim status	Claim Status
Generate Reports	Authorized User	Report criteria	Process available records	System Report
Logout	All Users	Logout request	End user session	Login Page

Figure 3.3: Event Table

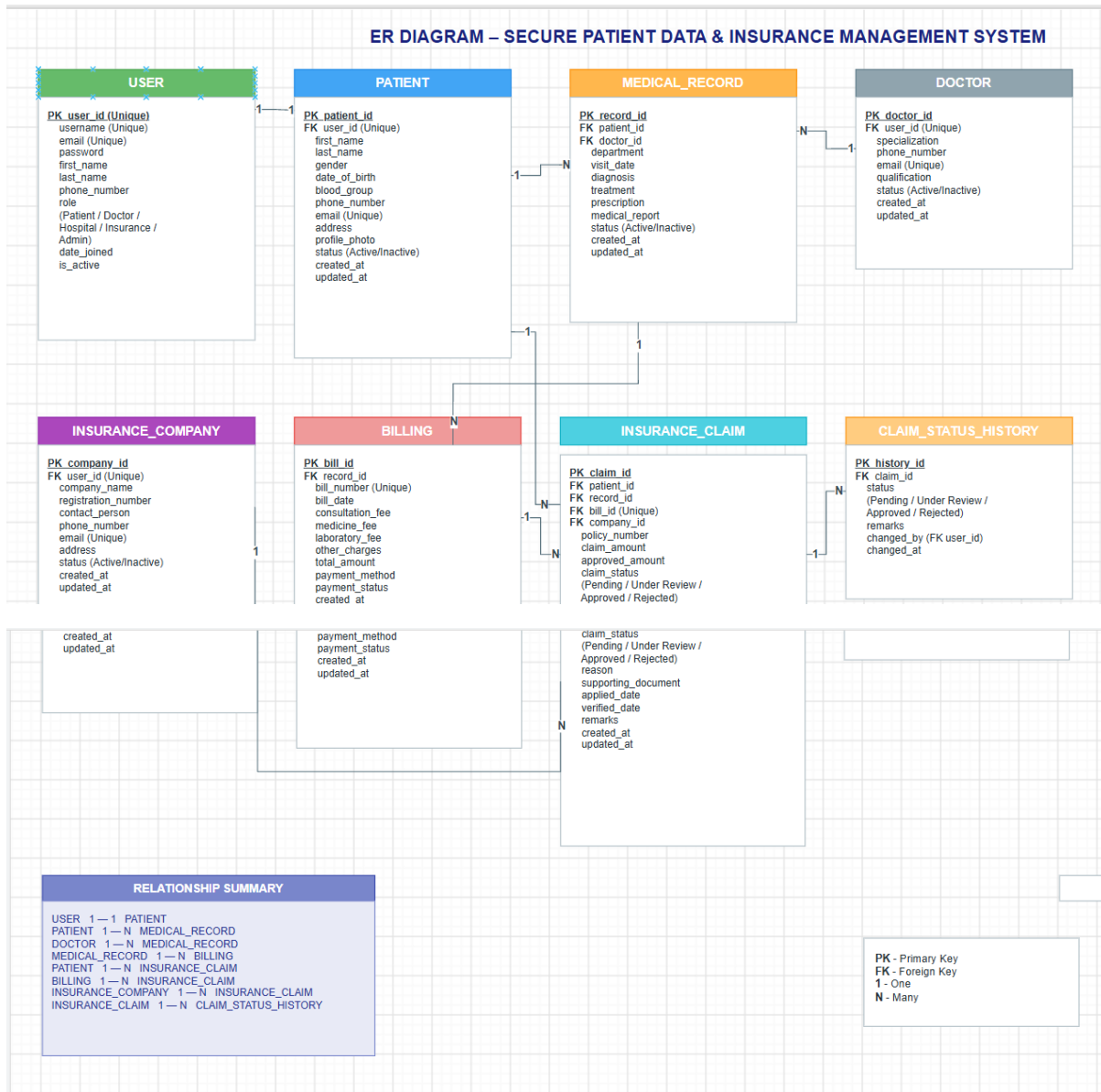


Figure 3.4: ER Diagram

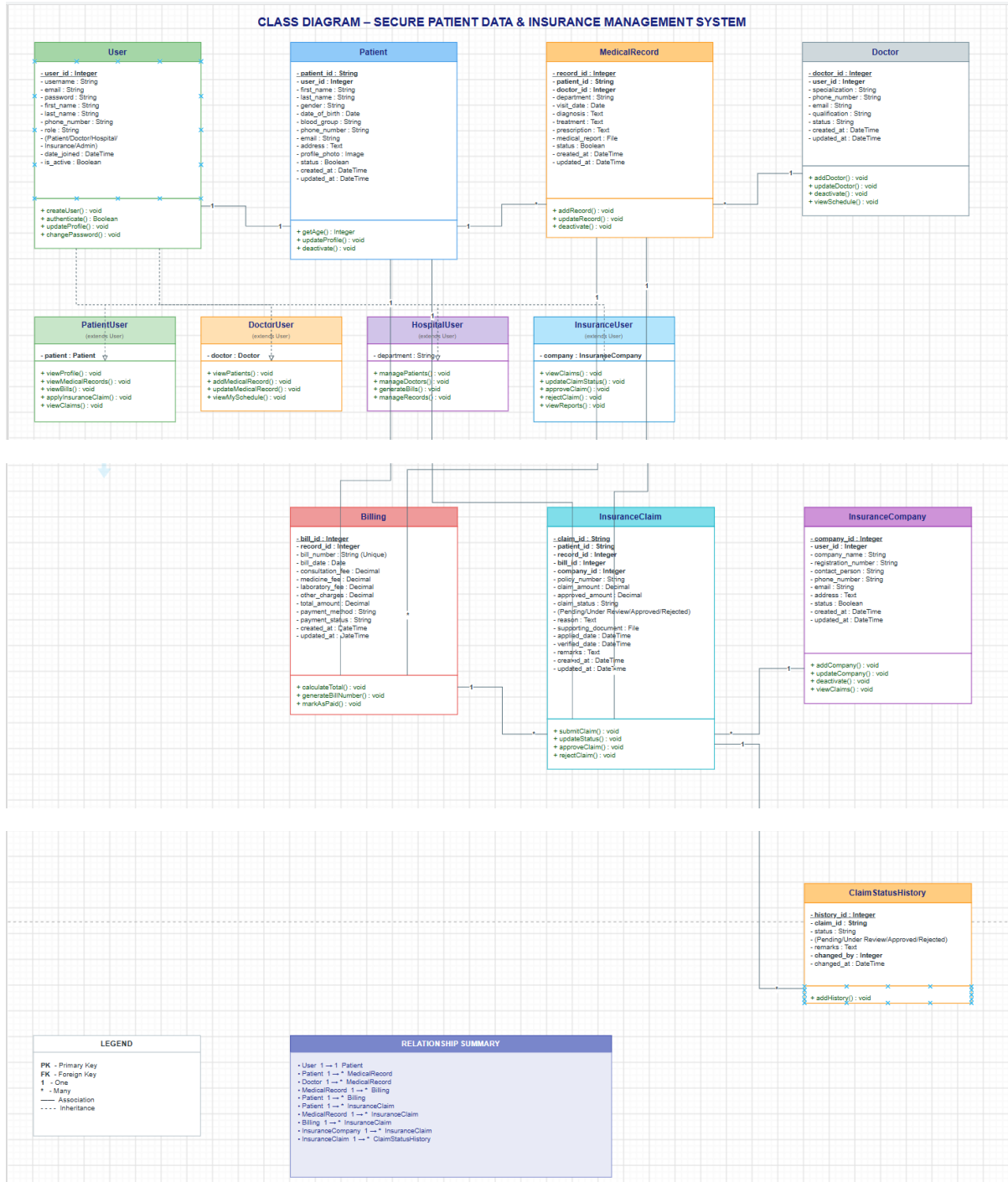


Figure 3.5: Class Diagram

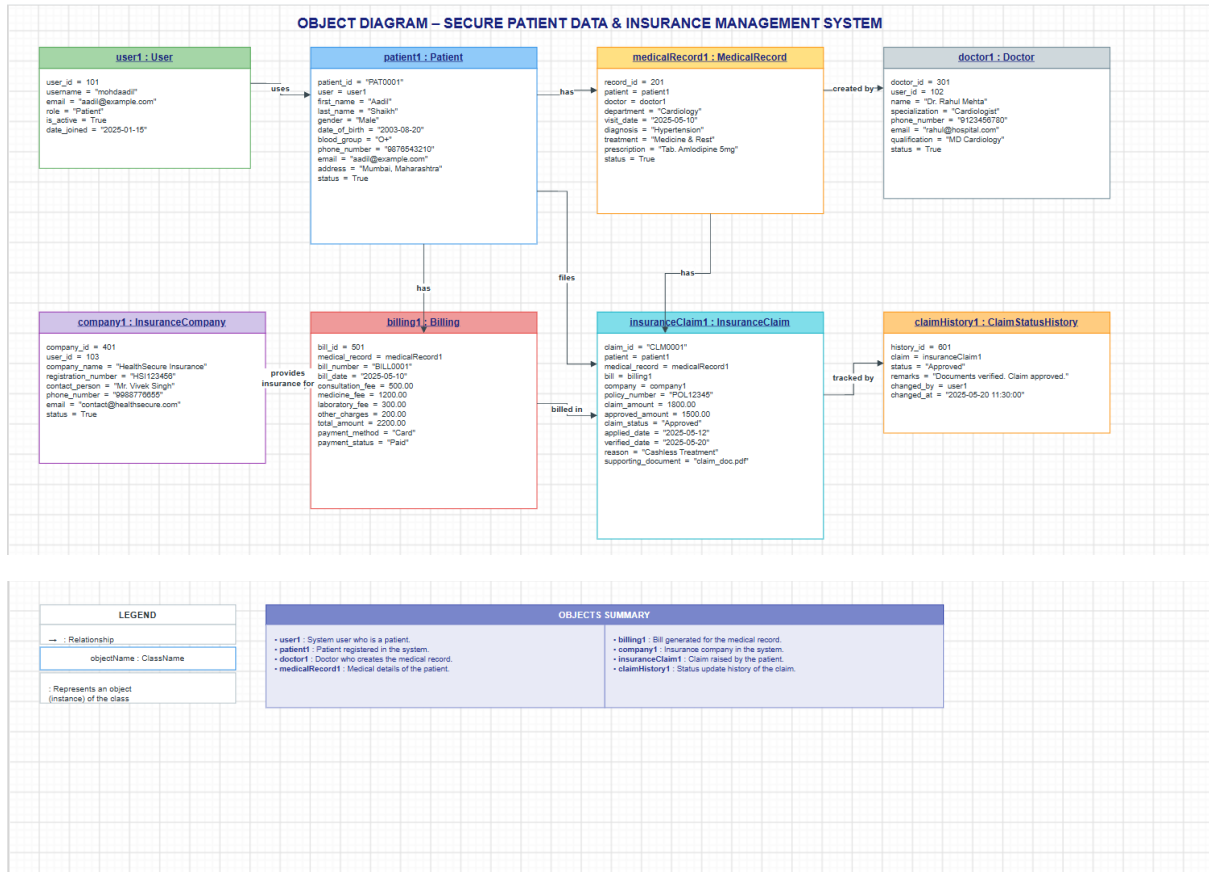


Figure 3.6: Object Diagram

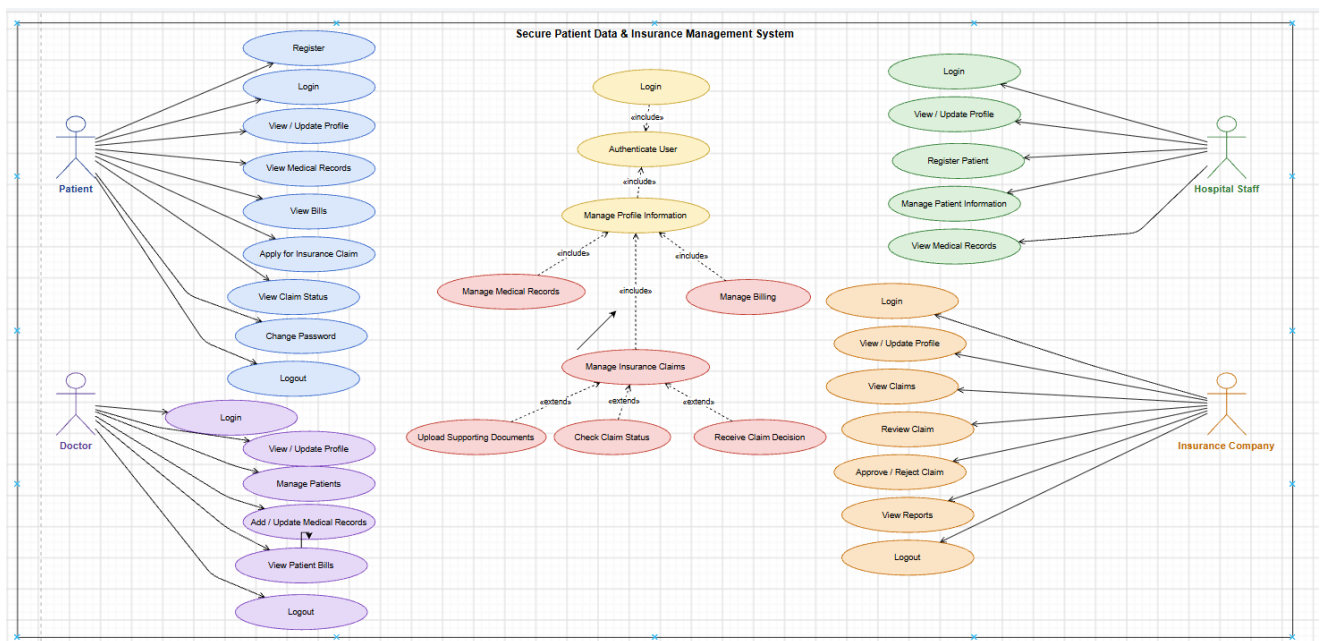


Figure 3.7: Use Case Diagram

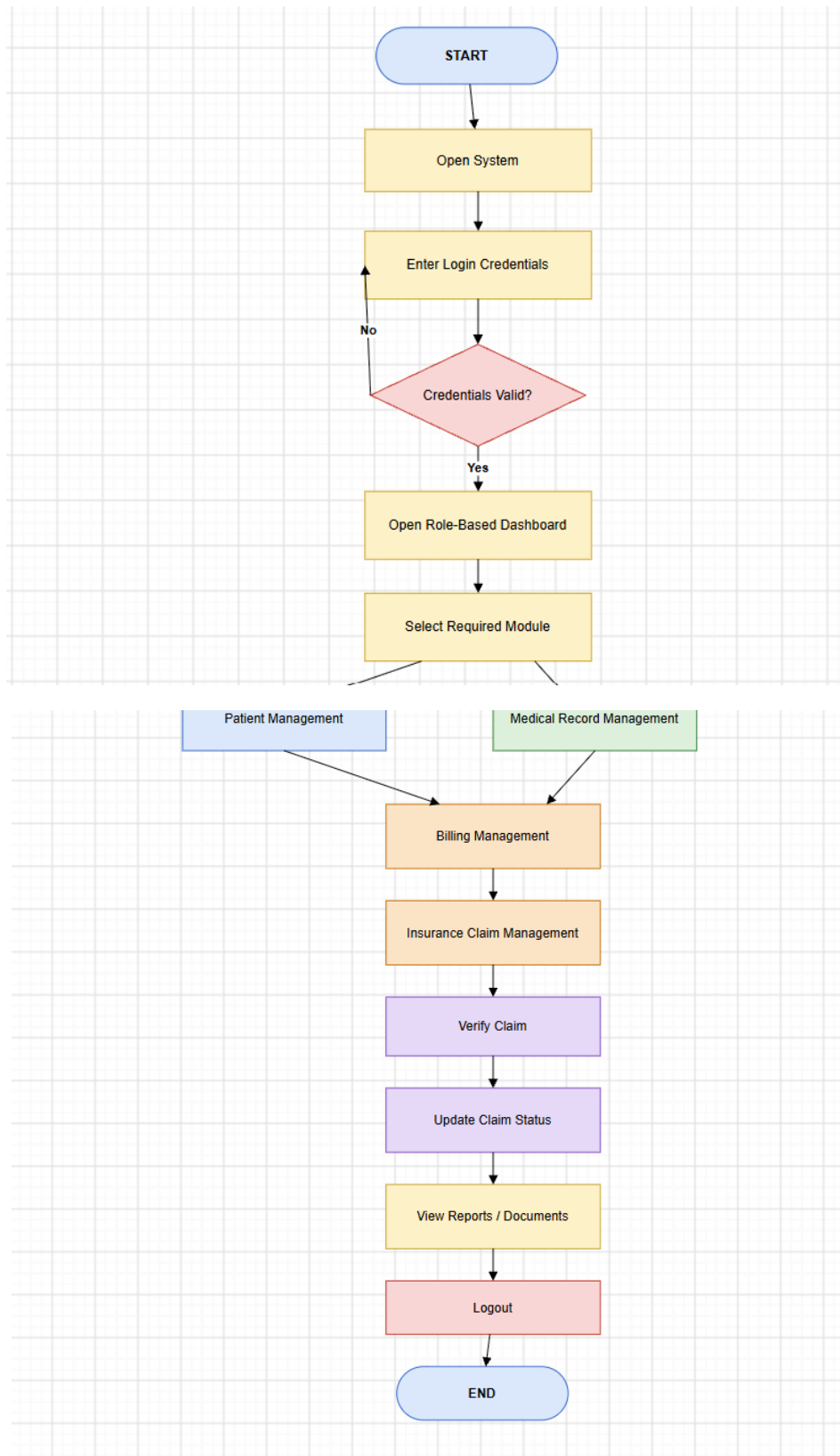


Figure 3.8: Activity Diagram

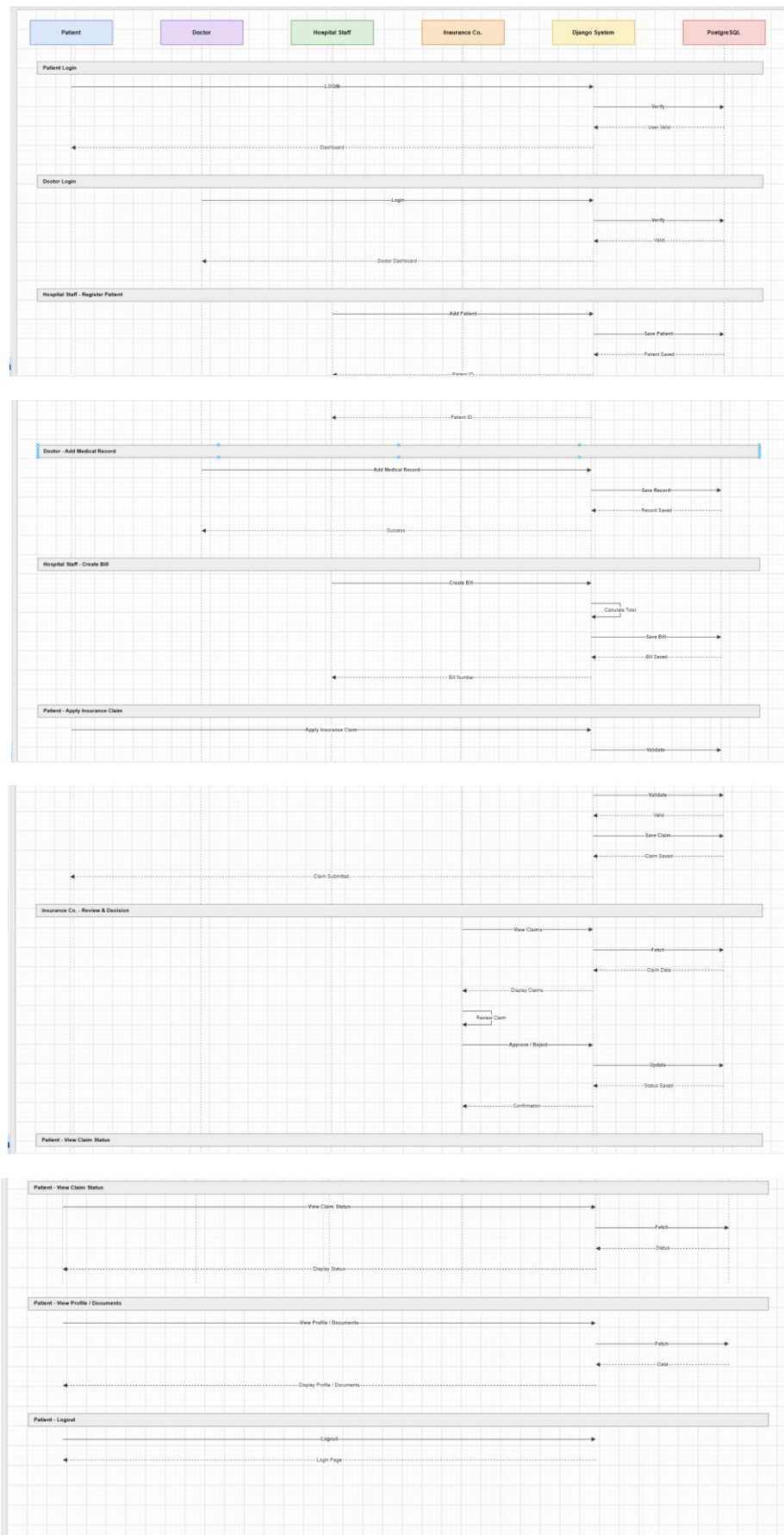


Figure 3.9: Sequence Diagram

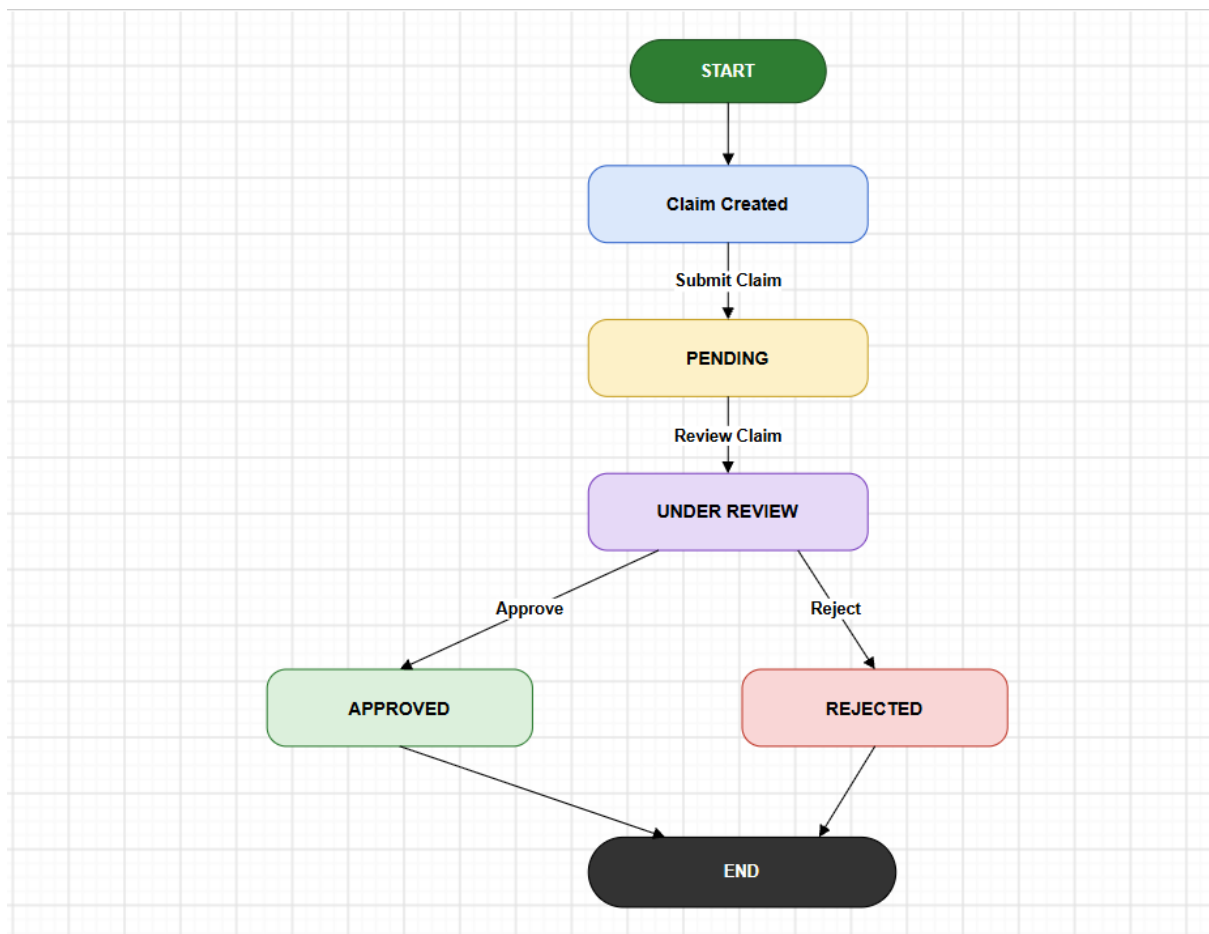


Figure 3.10: State Diagram of Insurance Claim Management

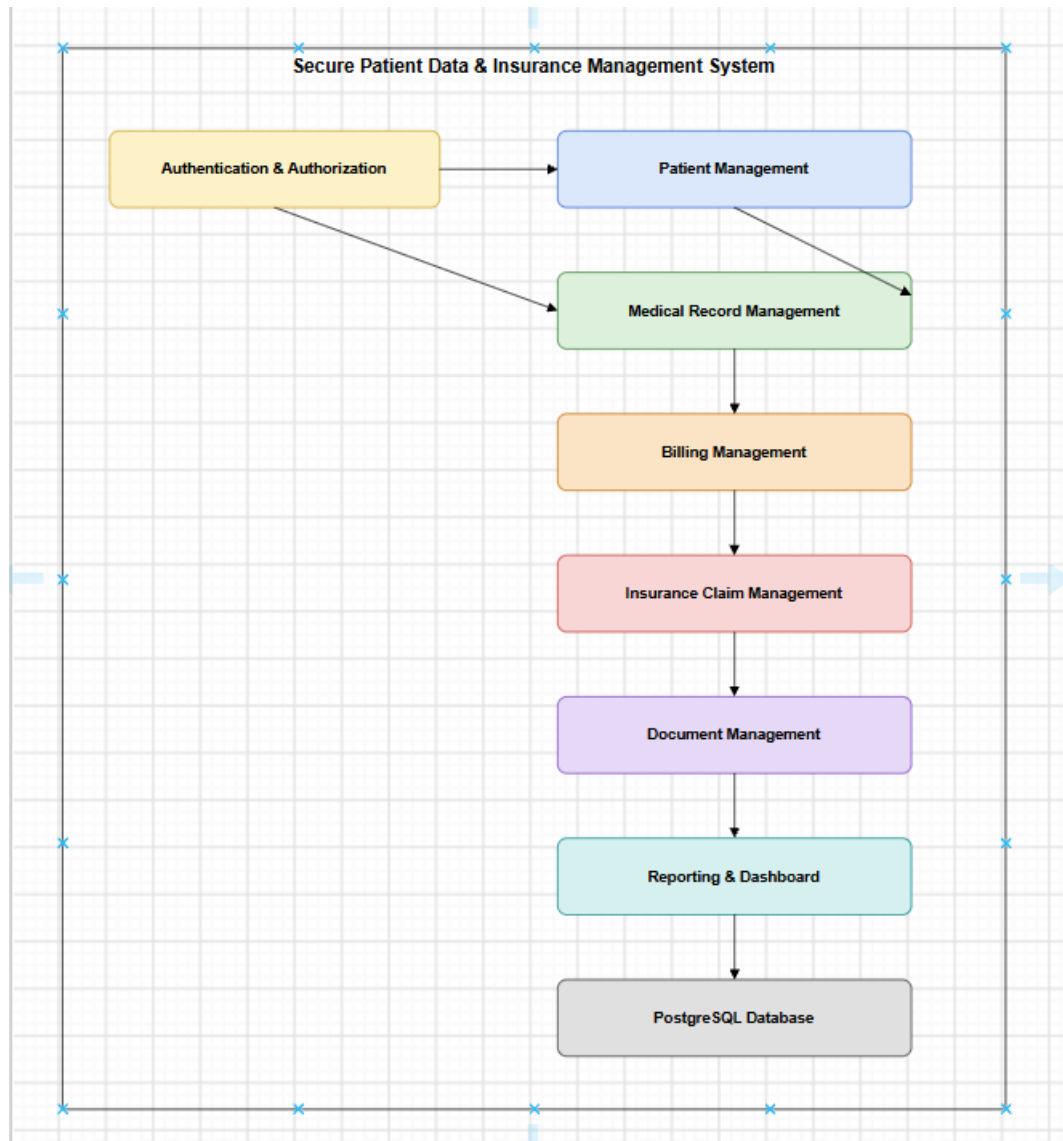


Figure 3.11: Package Diagram

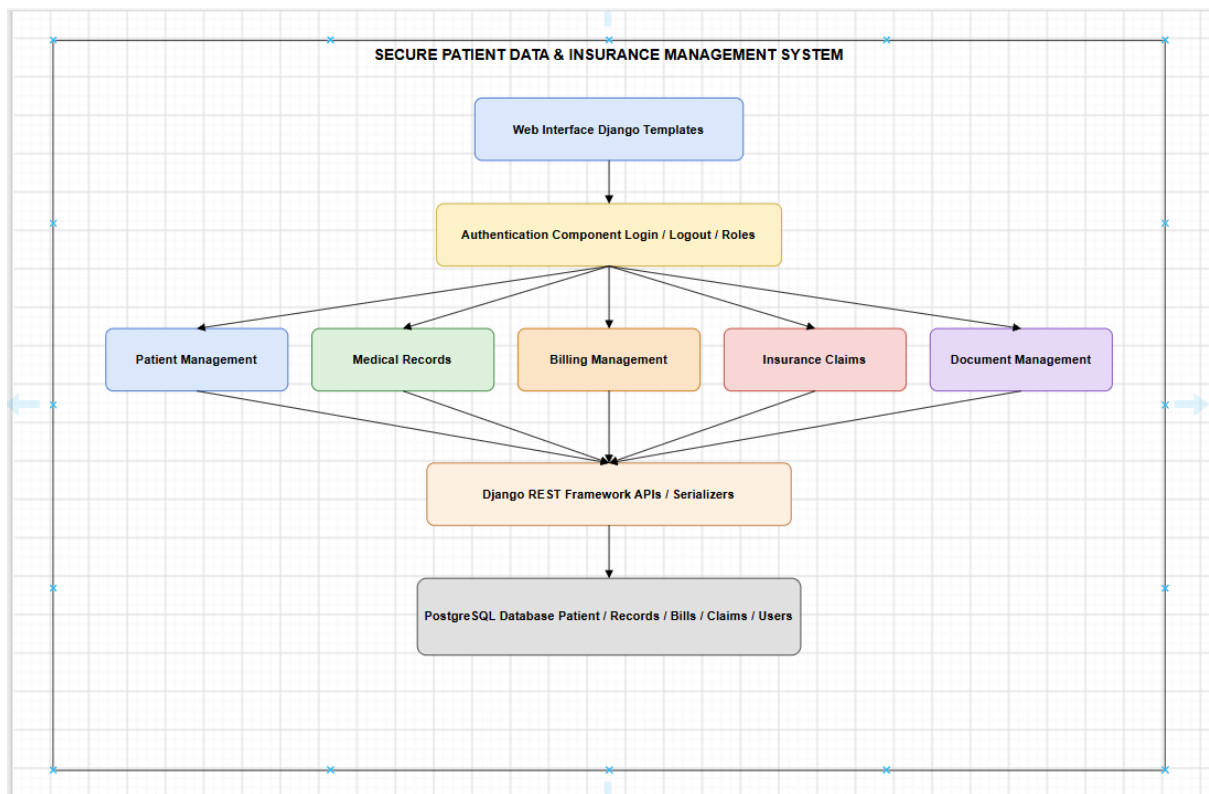


Figure 3.12: Component Diagram

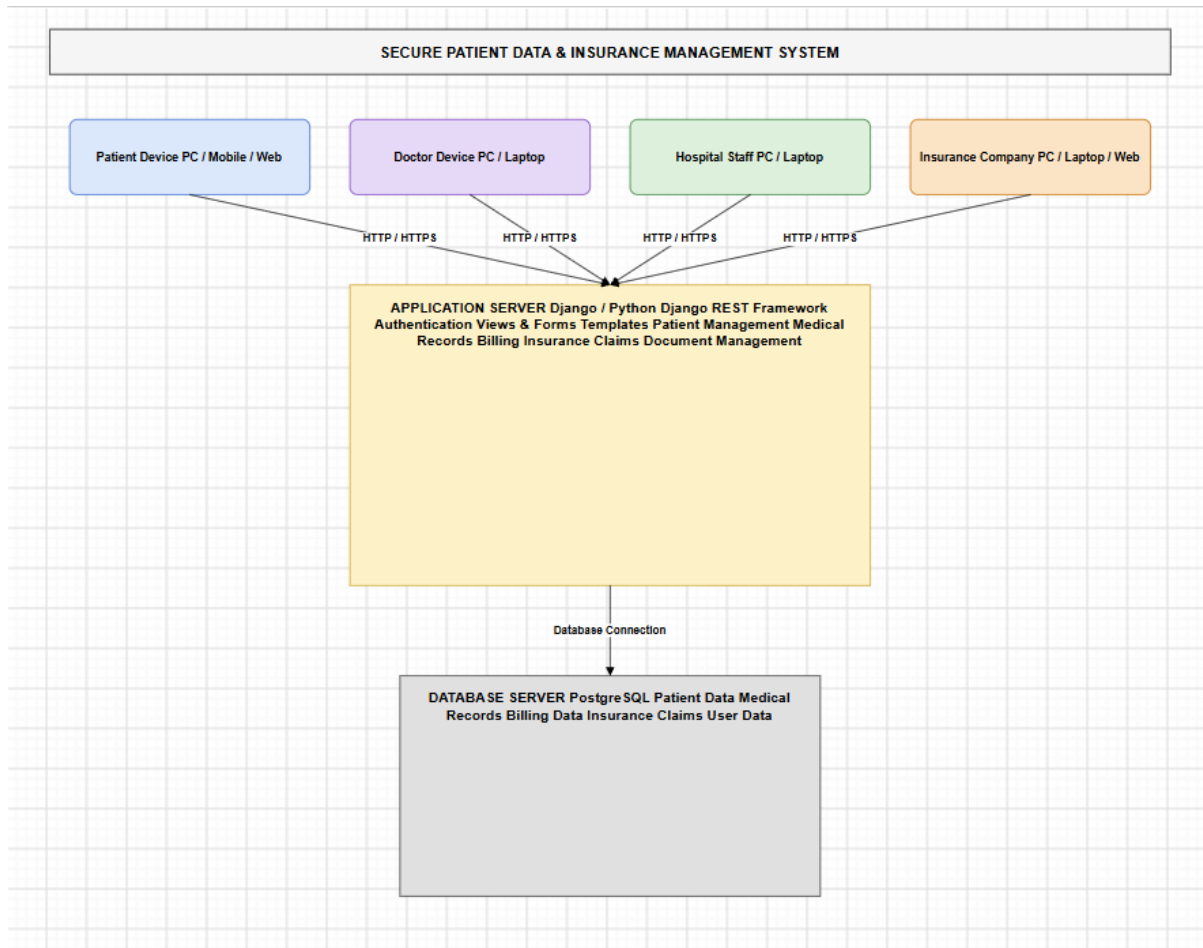


Figure 3.13: Deployment Diagram

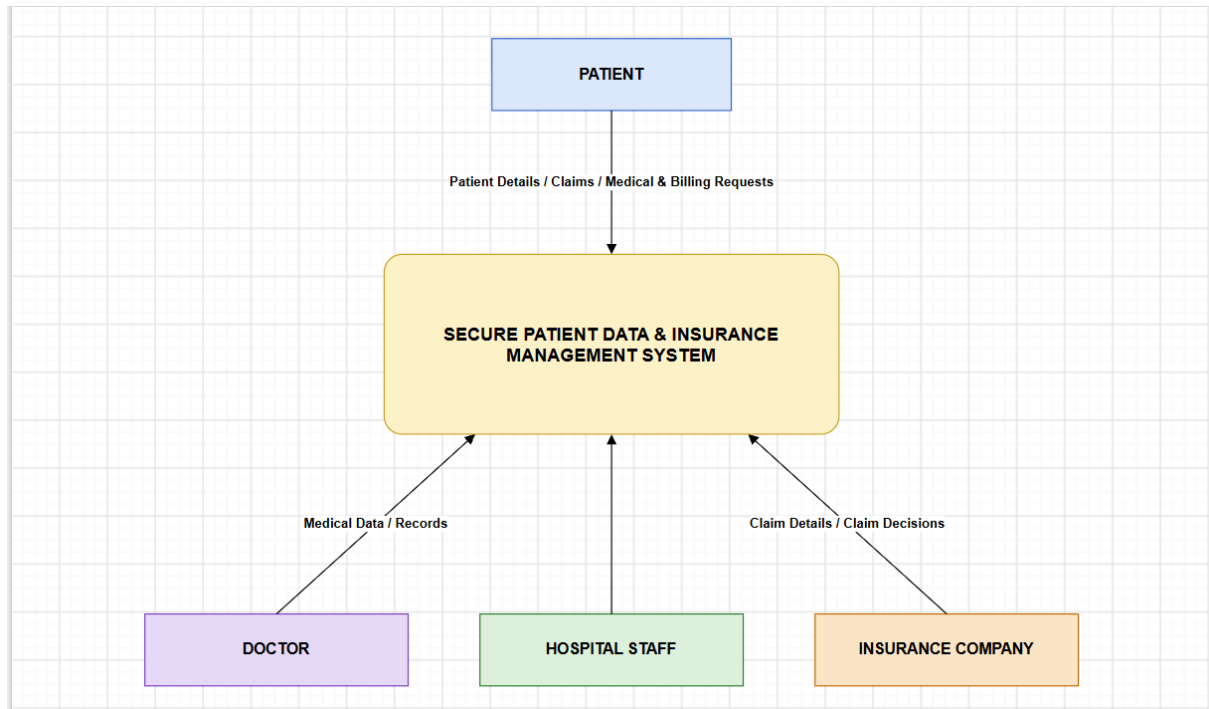


Figure 3.14: Data Flow Level 0 Diagram

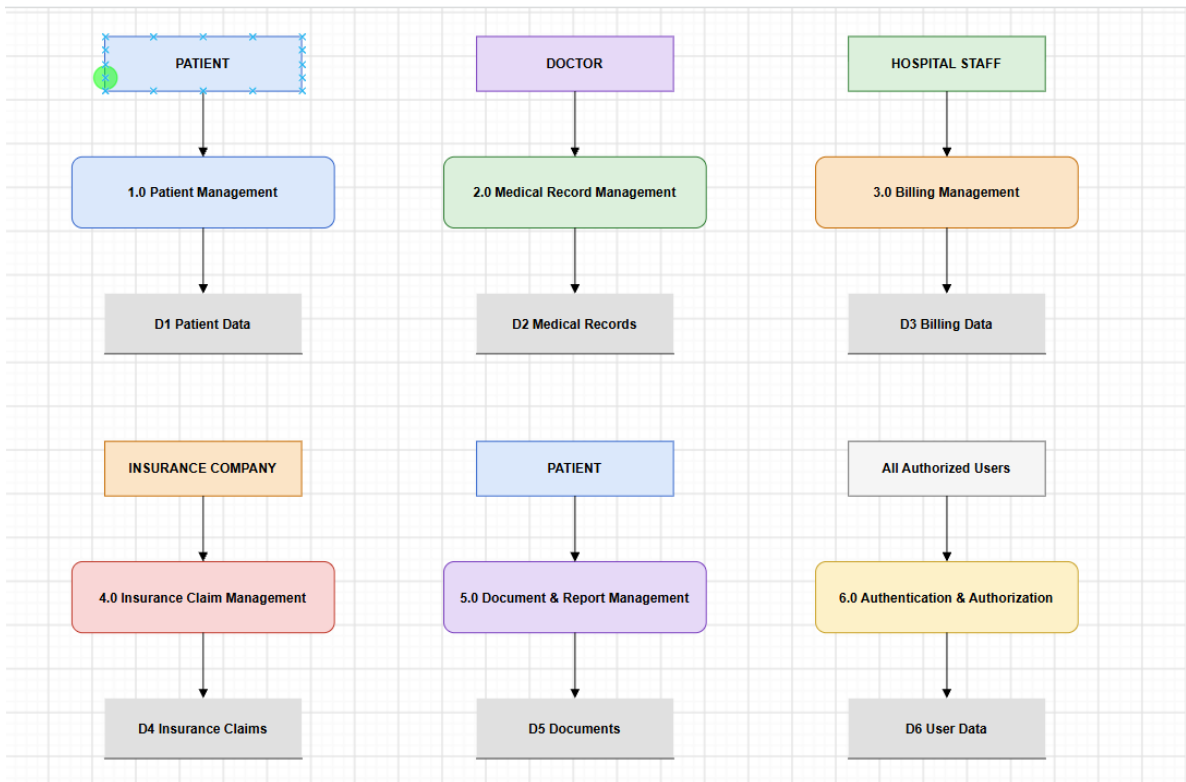


Figure 3.14.1: Data Flow Level 1 Diagram

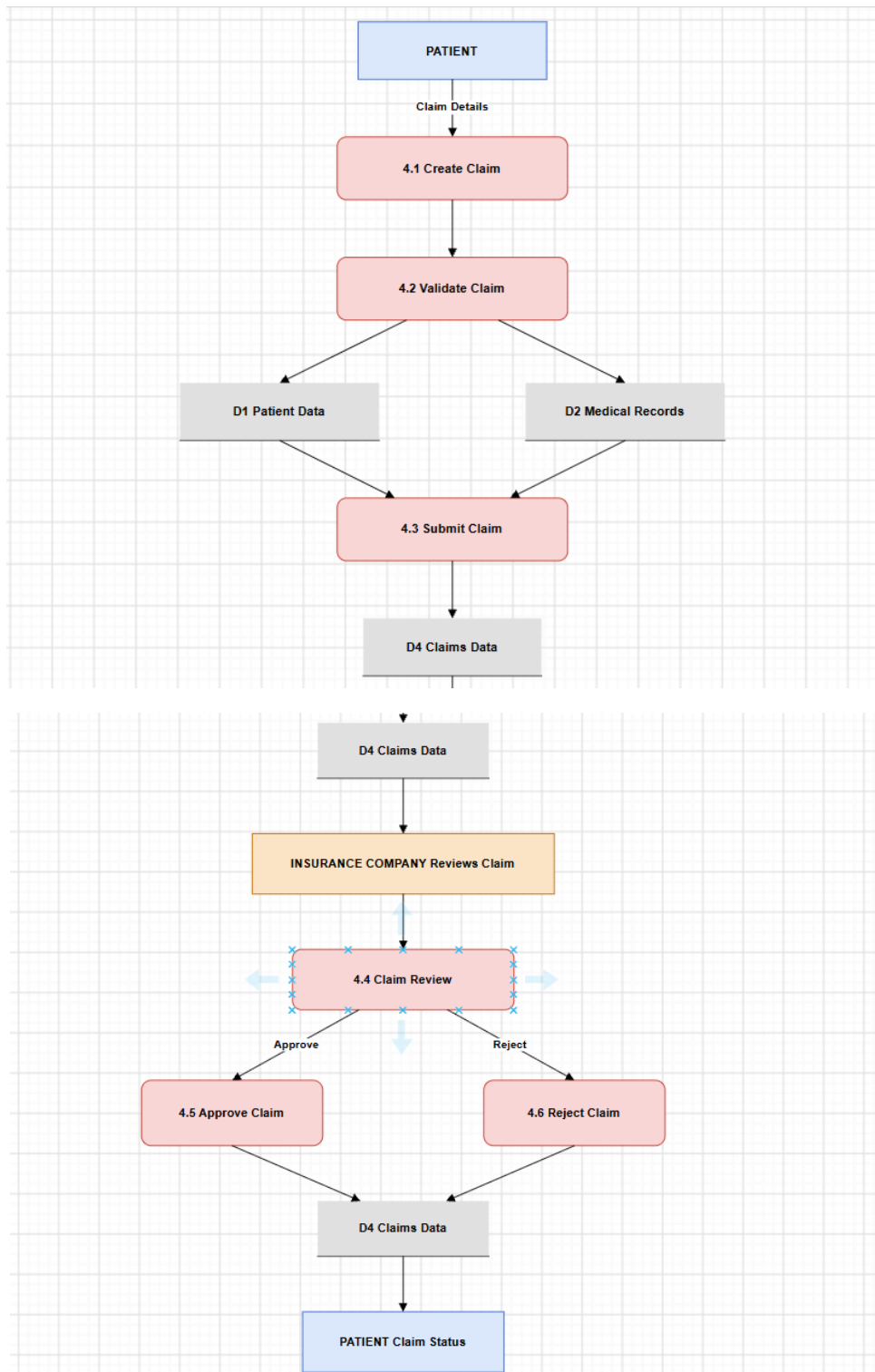


Figure 3.14.2: Data Flow Level 2 Diagram

CHAPTER 4: SYSTEM DESIGN

4.1 Basic Modules

The system is divided into different modules so that each part of the application can perform a specific task.

- **Patient Management:** Manages patient registration, profiles, and personal information.
- **Medical Record Management:** Stores diagnosis, treatment details, prescriptions, and medical reports.
- **Doctor Management:** Allows doctors to view patients and manage their medical records.
- **Billing Management:** Creates and manages hospital bills and generates downloadable PDF bills.
- **Insurance Management:** Handles insurance claim submission, verification, approval, and rejection.
- **User Management:** Provides login and role-based access for patients, doctors, hospital staff, billing staff, and insurance companies.
- **Dashboard and Reports:** Displays important information and claim statistics for authorized users.

4.2 Data Design

The database is designed to store and organize patient, medical, billing, insurance, and user information. PostgreSQL is used as the database for the project.

4.2.1 Schema Design

The main database entities include:

- **User** – Stores login and authentication information.
- **Patient** – Stores patient personal and profile information.
- **MedicalRecord** – Stores diagnosis, treatment, prescription, and medical information.
- **Billing** – Stores bill details, charges, payment information, and bill status.
- **InsuranceCompany** – Stores insurance company details.
- **InsuranceClaim** – Stores claim details, policy information, claim amount, status, and supporting documents.

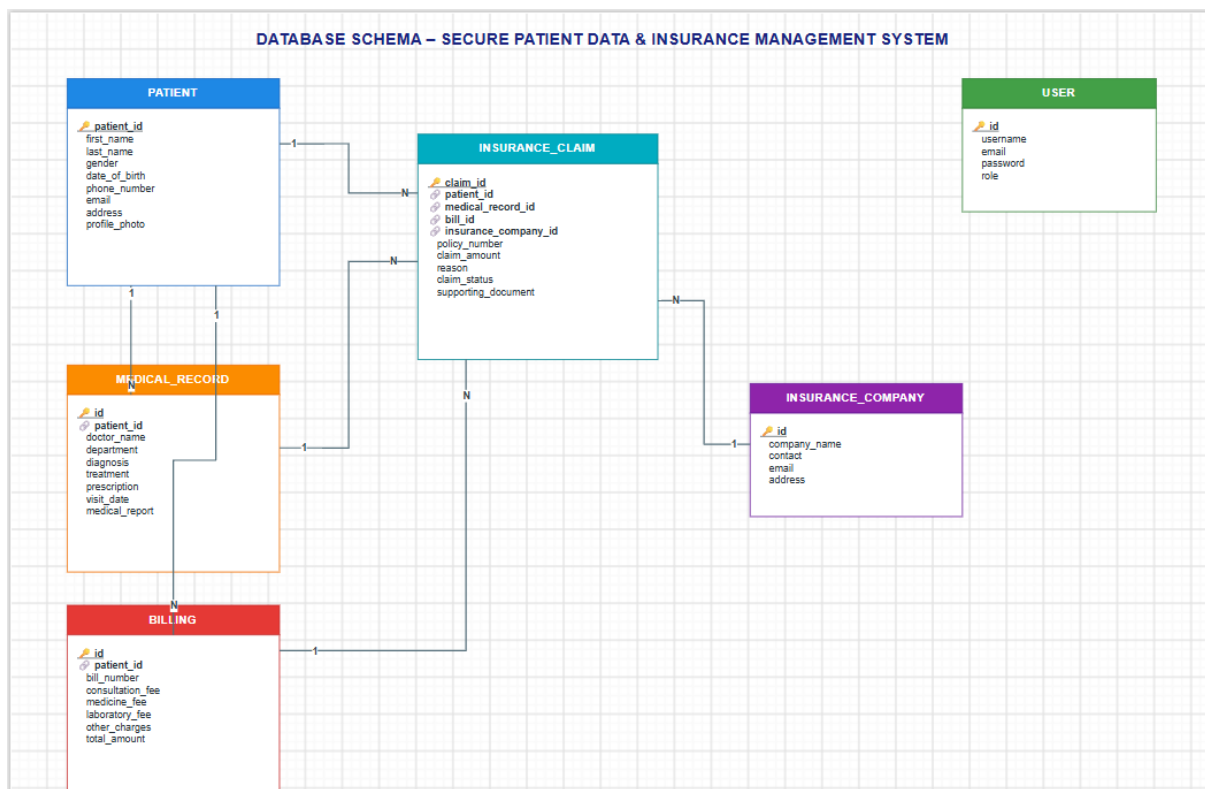


Figure 4.2.1: Database Schema Design

4.2.2 Data Integrity and Constraints

Data integrity ensures that the information stored in the Secure Patient Data & Insurance Management System remains accurate, consistent, and reliable. The system uses Django model validations and database constraints to prevent invalid, incomplete, or duplicate data.

The main data integrity measures used in the project are:

1. **Primary Key Constraint:**
Each database record has a unique primary key that identifies the record. This prevents two records from having the same primary key.
2. **Foreign Key Constraint:**
Foreign keys are used to establish relationships between tables. For example, medical records and billing records are connected to the appropriate patient.
3. **Unique Constraint:**
Important fields such as patient ID, claim ID, and email are maintained as unique where required. This helps prevent duplicate records.
4. **Required Fields:**
Important information cannot be left empty when it is required for creating a record. This ensures that essential patient, billing, and claim information is available.
5. **Patient and Medical Record Validation:**
The system checks that medical records and related information belong to the correct patient before they are used for further processing.
6. **Insurance Claim Validation:**
The claim amount is checked against the total bill amount. A claim cannot be submitted with an amount greater than the associated bill.
7. **Status Validation:**
Insurance claims can only use the defined statuses such as Pending, Under Review, Approved, and Rejected. This keeps claim processing consistent.
8. **Model-Level Validation:**
Django's `full_clean()` method is used before saving important model data so that validation rules are applied before information is stored.
9. **Relationship Integrity:**
The Insurance Claim is connected with the Patient, Medical Record, Billing, and Insurance Company. This ensures that claims are associated with valid and relevant records.
10. **Controlled Access:**
Role-based permissions prevent unauthorized users from modifying or accessing records that do not belong to their role.

4.3 Procedural Design

Procedural design describes the step-by-step working of the major functions of the system. It explains how user requests are processed, how data is validated, and how different modules work together.

The main procedures of the system include patient registration, medical record management, billing, insurance claim processing, authentication, and document management.

4.3.1 Logic Diagrams

Logic diagrams are used to represent the flow of different processes in the system. They help explain how the system takes input, processes the information, performs validation, and produces the required output.

The major logic flows of the project are:

Patient Management:

Login → Patient Registration → Validate Details → Save Patient Data → Generate Patient ID → Display Patient Profile

Medical Record Management:

Doctor Login → Select Patient → Enter Medical Details → Validate Data → Save Medical Record → Display Record

Billing Management:

Select Medical Record → Enter Charges → Calculate Total Amount → Generate Bill Number → Save Bill → Generate PDF Bill

Insurance Claim Management:

Patient/Staff → Select Medical Record and Bill → Enter Policy Details → Submit Claim → Validate Claim → Insurance Company Review → Approve/Reject Claim

4.3.2 Data Structures

The system uses Django models and PostgreSQL tables to store and manage data. Each major entity is represented using a separate model.

The main data structures are:

- **Patient** – Stores patient ID, personal details, contact information, blood group, date of birth, and profile photo.
- **MedicalRecord** – Stores diagnosis, symptoms, treatment, prescription, doctor, department, and medical reports.
- **Billing** – Stores bill number, different charges, total amount, payment status, and bill date.

- **InsuranceCompany** – Stores company details, registration number, contact details, and login information.
- **InsuranceClaim** – Stores claim ID, patient, medical record, bill, policy number, claim amount, approved amount, status, remarks, and supporting documents.

Relationships such as **ForeignKey** and **OneToOneField** are used to connect related records and avoid unnecessary duplication of data.

4.3.3 Algorithms Design

Algorithms are designed to perform important operations in a systematic way.

Algorithm 1: Patient Registration

1. Enter patient details.
2. Validate name, phone number, email, and date of birth.
3. Generate a unique Patient ID.
4. Create the patient's login account.
5. Display the patient profile.

Algorithm 2: Bill Generation

1. Select the medical record.
2. Enter consultation, medicine, laboratory, and other charges.
3. Check that charges are not negative.
4. Calculate the total bill amount.
5. Save the billing record.
6. Generate a unique bill number.
7. Generate the bill PDF.

Algorithm 3: Insurance Claim Processing

1. Select patient, medical record, and bill.
2. Enter policy number and claim amount.
3. Validate the claim details.
4. Check that the claim amount does not exceed the bill amount.
5. Insurance company reviews the claim.
6. Update the claim as Pending, Under Review, Approved, or Rejected.
7. Store the approved amount and remarks.

4.4 User Interface Design

The User Interface of the **Secure Patient Data & Insurance Management System** is designed to be simple, clear, and easy to use. Different dashboards are provided according to the user's role, such as **Patient, Doctor, Hospital Staff, and Insurance Company**.

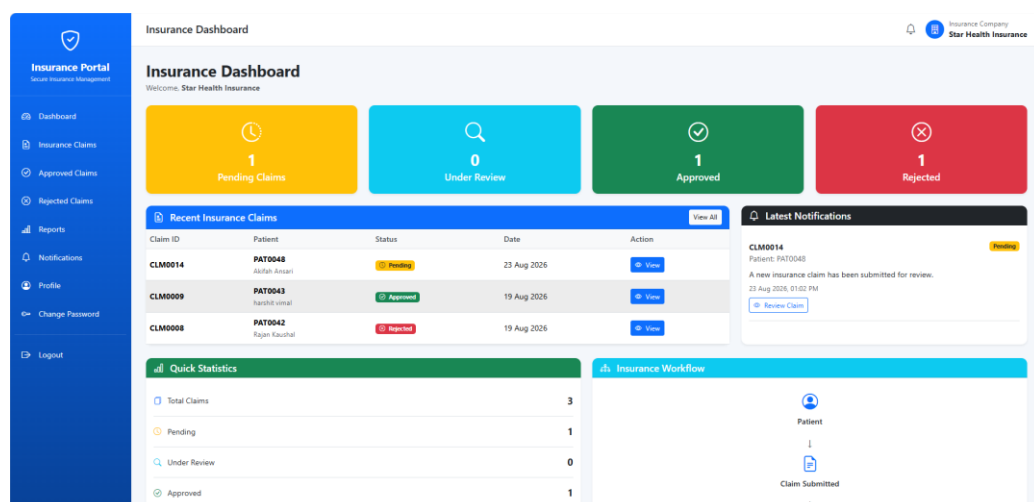
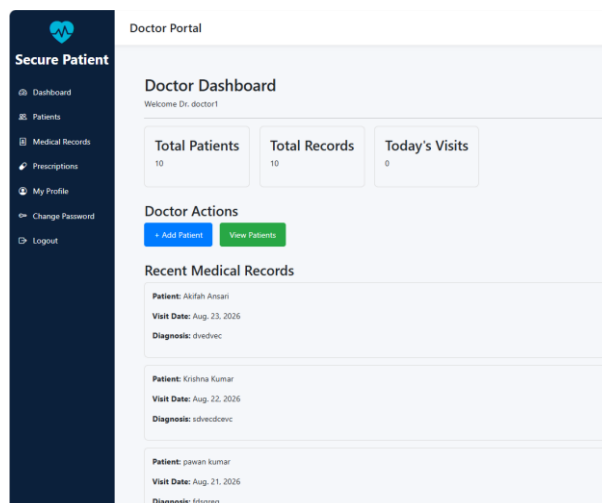
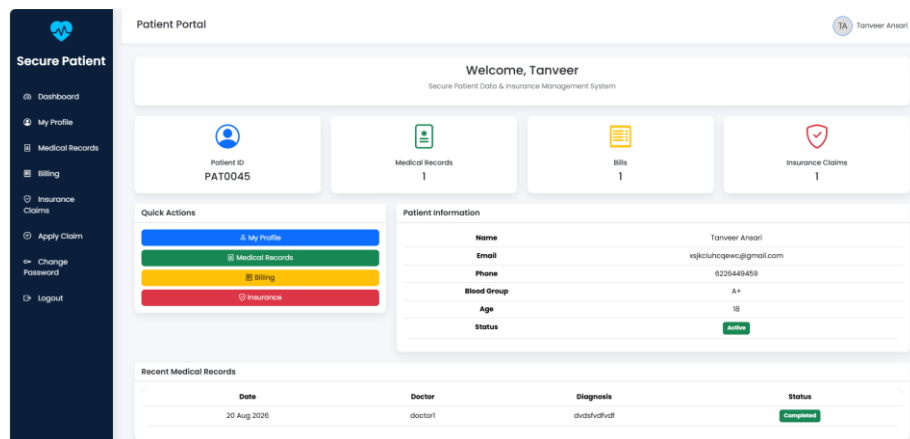
Main Interface Screens

1. **Login Page**
Allows authorized users to securely log in using their username/email and password.
2. **Patient Dashboard**
Provides access to patient profile, medical records, bills, and insurance claims.
3. **Doctor Dashboard**
Allows doctors to view patients, add medical records, manage prescriptions, and update treatment details.
4. **Hospital/Billing Dashboard**
Used for managing patient information and creating, editing, and viewing bills.
5. **Insurance Dashboard**
Allows insurance users to view claims and manage claims based on their status such as **Pending, Under Review, Approved, and Rejected**.
6. **Profile Page**
Allows users to view and manage their personal information.
7. **Medical Records Page**
Displays patient diagnosis, treatment, prescriptions, and medical reports.
8. **Billing Page**
Displays bill details and total amount and provides access to generated bills.
9. **Insurance Claim Page**
Allows patients to submit claims and insurance companies to review and process them.
10. **Reports and Notifications**
Provides claim information, system updates, and important notifications.

UI Design Features

- Simple and user-friendly layout
- Role-based dashboards
- Navigation menus for different modules
- Responsive Bootstrap-based design

- Tables for displaying records
- Forms for data entry
- Status badges for insurance claims
- Buttons for viewing, editing, and managing records
- Consistent design across different user portals



4.5 Security Issues

Since the system handles sensitive patient, medical, billing, and insurance information, security is an important part of the system design. The following security measures are used:

- **User Authentication:** Only registered users can log in to the system using valid credentials.
- **Role-Based Access Control:** Patients, doctors, hospital staff, and insurance companies have access only to the features required for their roles.
- **Password Protection:** User passwords are securely managed using Django's authentication system.
- **Restricted Patient Records:** Medical and personal information is accessible only to authorized users.
- **Company-Specific Access:** Insurance companies can view and manage claims related to their own organization.
- **Form Validation:** User inputs are validated before being stored in the database to reduce invalid or harmful data.
- **Document Security:** Uploaded medical reports, bills, and insurance documents are handled through the application's controlled media system.
- **CSRF Protection:** Django's CSRF protection is used to help prevent unauthorized requests through web forms.
- **Session Security:** Login sessions are managed by Django so that authenticated users can access protected pages securely.
- **Database Integrity:** Primary keys, foreign keys, unique fields, and validation rules help prevent inconsistent or incorrect records.

4.6 Test Cases Design

Test cases are prepared to check whether each module works as expected. The testing covers both normal and incorrect inputs.

Some important test cases include:

Test Case	Expected Result
Valid user login	User is redirected to the correct dashboard
Invalid login	Error message is displayed
Add patient	Patient record is created successfully
Add medical record	Record is saved with the correct patient
Generate bill	Bill is created and PDF can be downloaded
Submit insurance claim	Claim is created with Pending status
Approve claim	Claim status changes to Approved
Reject claim	Claim status changes to Rejected
Insurance company login	Company sees only its own claims
Upload document	Supporting document is stored successfully
Search patient by name	Matching patient records are displayed
Edit patient information	Updated details are saved and reflected immediately
Insurance company views claim without permission	Access is denied with an appropriate message
Generate report for approved claims	Report lists only claims with Approved status

CHAPTER 5: IMPLEMENTATION AND TESTING

5.1 Implementation Approaches

The Secure Patient Data & Insurance Management System was developed as a web-based application using Python and Django, with PostgreSQL used for storing and managing the system data.

The implementation was carried out module by module. First, the database structure was created using Django models. After that, views, forms, templates, and URL routing were developed according to the requirements of each module.

The main implementation areas include:

- Patient Management
- Medical Record Management
- Billing Management
- Insurance Claim Management
- Authentication and Role-Based Access
- Document Management
- Dashboard and Reporting

Django's built-in authentication and permission features are used to control access to different parts of the system. Validation is also applied to reduce incorrect or incomplete data.

The modules were tested individually first and then integrated together to check whether the complete system works properly.

5.2 Coding Details and Code Efficiency

The system is developed using a modular coding approach. Different parts of the application are handled using Django Models, Views, Forms, Serializers, Templates, and URL routing.

Models are responsible for defining the database structure, while views handle the main application logic. Forms are used to collect and validate user input, and templates are used to display information to users.

The project uses Django REST Framework for selected API-based operations. This makes it easier to manage patient, medical record, billing, and insurance data through structured APIs.

The important coding areas of the project are described below.

Database Models

Django models are used to create and manage the database structure. Important models include:

- Patient
- MedicalRecord
- Billing
- InsuranceClaim
- InsuranceCompany

Relationships such as ForeignKey and OneToOneField are used to connect related records and avoid unnecessary duplication of data.

Patient Management

The Patient Management module allows authorized users to add, view, search, update, and manage patient information.

The patient information includes details such as:

- Patient ID
- Name
- Gender
- Date of Birth
- Blood Group
- Phone Number
- Email

- Address
- Emergency Contact
- Profile Photo

Django REST Framework ViewSets are used to provide API operations for patient records.

Medical Record Management

The Medical Record module is used to maintain information related to a patient's medical history.

It stores information such as:

- Patient details
- Doctor name
- Department
- Diagnosis
- Treatment
- Prescription
- Visit date
- Medical reports

Search and filtering features help authorized users find the required medical records easily.

Billing Management

The Billing module manages different types of hospital charges such as consultation fees, medicine charges, laboratory charges, and other expenses.

The total amount is calculated automatically from the individual charges. A unique bill number is also generated for every bill.

The system also provides PDF bill generation, allowing users to keep a digital copy of the bill.

Insurance Claim Management

The Insurance Claim module allows patients to submit insurance claims using their medical and billing information.

A claim can contain:

- Patient information
- Medical record
- Bill information

- Insurance company
- Policy number
- Claim amount
- Claim reason
- Supporting documents

The claim moves through different stages such as:

Pending → Under Review → Approved / Rejected

The system also checks whether the claim amount is within the bill amount and whether the selected medical record and bill belong to the correct patient.

Authentication and Role-Based Access

Django authentication is used to provide secure login to the system.

Different users have different responsibilities and access levels, such as:

- Patient
- Doctor
- Hospital Staff
- Insurance Company
- Administrator

This prevents users from accessing functions that are not related to their role.

Form Validation

Forms are used to validate information before it is stored in the database.

Validation is applied to important fields such as:

- Phone number
- Email
- Date of birth
- Claim amount
- Required fields
- Uploaded documents

Model-level validation is also used where required to prevent incorrect data from entering the database.

URL Routing

Django URL routing connects the requested URL with the appropriate view.

Separate URLs are provided for different sections such as:

- Patient
- Doctor
- Hospital
- Billing
- Insurance
- Authentication

Django REST Framework routers are also used for API endpoints.

5.2.1 Code Efficiency

The project follows a modular and reusable coding approach to keep the application easier to maintain and update.

The following techniques are used to improve code efficiency:

- **Code Reusability:** Common models, forms, templates, and functions are reused instead of writing the same code repeatedly.
- **Efficient Database Queries:** Django ORM is used to retrieve the required records from PostgreSQL.
- **Reduced Database Calls:** Unnecessary database operations are avoided wherever possible.
- **Database Relationships:** ForeignKey and OneToOneField relationships reduce duplicate data.
- **Input Validation:** User input is checked before processing to reduce errors.
- **Query Filtering:** Only active or relevant records are retrieved when required.
- **Search and Ordering:** Search and sorting are handled using Django and REST Framework features.
- **Modular Code:** Different application features are separated into suitable modules.
- **Reusable Templates:** Common page structures can be reused across different screens.
- **Role-Based Permissions:** Users only access the functions allowed for their role.

These practices make the code easier to understand, maintain, debug, and improve in the future.

Code:

```
from rest_framework import filters
from rest_framework import viewsets
from .serializers import PatientSerializer
from .pagination import PatientPagination
from rest_framework.permissions import IsAuthenticated
from .models import (Patient, MedicalRecord, Billing, InsuranceClaim, InsuranceCompany,)
from .serializers import (MedicalRecordSerializer, BillingSerializer,)
from rest_framework.permissions import DjangoModelPermissions
from django.http import HttpResponse
from .pdf import generate_bill_pdf
from django.shortcuts import render, redirect, get_object_or_404
from django.contrib.auth import authenticate, login, logout, update_session_auth_hash
from django.contrib.auth.decorators import login_required
from django.contrib.auth.forms import PasswordChangeForm
from .forms import (InsuranceClaimForm, PatientPasswordChangeForm, BillingForm,
MedicalRecordForm, PatientForm, PatientInsuranceClaimForm )
from django.contrib import messages
from django.utils import timezone
from django.db.models import Q, Sum, Exists, OuterRef
from datetime import date
from reportlab.pdfgen import canvas
from reportlab.lib.pagesizes import A4
from django.http import HttpResponseForbidden
from django.core.exceptions import ValidationError
from decimal import Decimal, InvalidOperation
from reportlab.lib.units import mm
from reportlab.pdfbase.ttfonts import TTFont
```

```

from reportlab.pdfbase import pdfmetrics
from reportlab.lib import colors
from reportlab.lib.colors import HexColor

class PatientViewSet(viewsets.ModelViewSet):

    queryset = Patient.objects.all()
    serializer_class = PatientSerializer
    permission_classes = [DjangoModelPermissions]
    search_fields = [ 'patient_id', 'first_name', 'last_name', 'phone_number', 'email' ]
    ordering_fields = ['patient_id', 'first_name', 'last_name', 'created_at', 'date_of_birth',]
    ordering = ['patient_id']

    def perform_destroy(self, instance):

        instance.status = False

        instance.save()

class MedicalRecordViewSet(viewsets.ModelViewSet):

    queryset = MedicalRecord.objects.filter(status=True)
    serializer_class = MedicalRecordSerializer
    filter_backends = [filters.SearchFilter, filters.OrderingFilter, ]
    search_fields = [

        "patient__patient_id",
        "patient__first_name",
        "patient__last_name",
        "doctor_name",
        "department",
        "diagnosis",
    ]
    ordering_fields = [

        "visit_date",
        "doctor_name",
        "department",

```

```

        "created_at",
    ]
    ordering = ["-visit_date"]
class BillingViewSet(viewsets.ModelViewSet):
    queryset = Billing.objects.all()
    serializer_class = BillingSerializer
    filter_backends = [ filters.SearchFilter, filters.OrderingFilter, ]
    search_fields = [
        "medical_record__patient__patient_id",
        "medical_record__patient__first_name",
        "medical_record__patient__last_name", ]
    ordering_fields = [ "bill_date", "total_amount", ]
    ordering = ["-bill_date"]
from rest_framework.decorators import api_view
def patient_login(request):
    if request.user.is_authenticated:
        return redirect("patient_dashboard")
    if request.method == "POST":
        username = request.POST.get("username")
        password = request.POST.get("password")
        user = authenticate(
            request,
            username=username,
            password=password,
        )
    if user is not None:
        login(request, user)
        return redirect("patient_dashboard")
    else:

```

```

        return render(
            request,
            "patient/login.html", {
                "error": "Invalid Username or Password" } )
    return render(
        request,
        "patient/login.html" )

```

@login_required

```

def patient_dashboard(request):
    patient = get_object_or_404(
        Patient,
        user=request.user )
    medical_count = MedicalRecord.objects.filter(
        patient=patient
    ).count()
    billing_count = Billing.objects.filter(
        medical_record__patient=patient
    ).count()
    claim_count = InsuranceClaim.objects.filter(
        patient=patient
    ).count()
    recent_records = MedicalRecord.objects.filter(
        patient=patient
    ).order_by("-created_at")[:5]
    return render(
        request,
        "patient/dashboard.html",
        {

```

```

        "patient": patient,
        "medical_count": medical_count,
        "billing_count": billing_count,
        "claim_count": claim_count,
        "recent_records": recent_records, }, )

```

```

@login_required

```

```

def patient_profile(request)

```

```

    patient = get_object_or_404(

```

```

        Patient,

```

```

        user=request.user )

```

```

    return render(

```

```

        request,

```

```

        "patient/profile.html",

```

```

        {

```

```

            "patient": patient,

```

```

        }

```

```

    )

```

```

@login_required

```

```

def patient_download_bill(request, pk):

```

```

    patient = get_object_or_404(

```

```

        Patient,

```

```

        user=request.user

```

```

    )

```

```

    bill = get_object_or_404(

```

```

        Billing,

```

```

        pk=pk,

```

```

        medical_record__patient=patient

```

```

    )

```

```

    response = HttpResponseRedirect(

```

```

        content_type="application/pdf"
    )
    response["Content-Disposition"] = (
        fattachment; filename="Bill_{bill.bill_number}.pdf"
    )
    pdf = canvas.Canvas(
        response,
        pagesize=A4
    )
    width, height = A4
    pdf.setFont("Helvetica-Bold", 20)
    pdf.drawString(
        50,
        height - 60,
        "SECURE PATIENT SYSTEM"
    )
    pdf.setFont("Helvetica-Bold", 16)
    pdf.drawString(
        50,
        height - 100,
        "MEDICAL BILL"
    )
    y = height - 150
    pdf.setFont("Helvetica", 11)
    pdf.drawString(
        50,
        y,
        f"Bill Number: {bill.bill_number}"
    )

```

```

y -= 25
pdf.drawString(
    50,
    y,
    f"Patient ID: {patient.patient_id}"
)
y -= 25
pdf.drawString(
    50,
    y,
    f"Patient Name: {patient.first_name} {patient.last_name}"
)
y -= 25
pdf.drawString(
    50,
    y,
    f"Bill Date: {bill.bill_date}"
)
y -= 50
pdf.setFont("Helvetica-Bold", 12)
pdf.drawString(
    50,
    y,
    "Billing Details"
)
y -= 30
pdf.setFont("Helvetica", 11)
pdf.drawString(
    70,

```

```

        y,
        f"Consultation Fee: Rs. {bill.consultation_fee}"
    )
    y -= 25
    pdf.drawString(
        70,
        y,
        f"Medicine Fee: Rs. {bill.medicine_fee}"
    )
    y -= 25
    pdf.drawString(
        70,
        y,
        f"Laboratory Fee: Rs. {bill.laboratory_fee}"
    )
    y -= 25
    pdf.drawString(
        70,
        y,
        f"Other Charges: Rs. {bill.other_charges}"
    )
    y -= 40
    pdf.setFont("Helvetica-Bold", 14)

    pdf.drawString(
        70,
        y,
        f"TOTAL: Rs. {bill.total_amount}"
    )

```

```

y -= 35
pdf.setFont("Helvetica", 11)
pdf.drawString(
    70,
    y,
    f"Payment Method: {bill.payment_method}"
)
y -= 25
pdf.drawString(
    70,
    y,
    f"Payment Status: {bill.payment_status}"
)
pdf.setFont("Helvetica-Oblique", 9)
pdf.drawString(
    50,
    50,
    "This is a computer-generated bill."
)
pdf.save()
return response

def insurance_login(request):
    if request.user.is_authenticated:
        return redirect("insurance_dashboard")
    if request.method == "POST":
        username = request.POST.get("username")
        password = request.POST.get("password")
        user = authenticate(
            request,

```

```

        username=username,
        password=password
    )
    if user is not None:
        if user.groups.filter(name="Insurance").exists():
            login(request, user)
            return redirect("insurance_dashboard")
        else:
            return render(
                request,
                "insurance/login.html",
                {
                    "error": "You are not authorized to access the Insurance Portal."
                }
            )
    return render(
        request,
        "insurance/login.html",
        {
            "error": "Invalid username or password."
        }
    )
    return render(
        request,
        "insurance/login.html"
    )
@login_required
def insurance_dashboard(request):
    company = get_object_or_404(

```

```

InsuranceCompany,
user=request.user
)
claims = InsuranceClaim.objects.select_related(
    "patient",
    "medical_record",
    "bill"
).filter(
    insurance_company=company
)
pending_claims = claims.filter(
    claim_status="Pending"
).count()
under_review_claims = claims.filter(
    claim_status="Under Review"
).count()
approved_claims = claims.filter(
    claim_status="Approved"
).count()
rejected_claims = claims.filter(
    claim_status="Rejected"
).count()
total_claims = claims.count()
recent_claims = claims.order_by(
    "-created_at"
)[:5]
notifications = claims.filter(
    claim_status="Pending"
).order_by(

```

```
    "-created_at"
  )[:5]
  return render(
    request,
    "insurance/dashboard.html",
    {
      "company": company,
      "total_claims": total_claims,
      "pending_claims": pending_claims,
      "under_review_claims": under_review_claims,
      "approved_claims": approved_claims,
      "rejected_claims": rejected_claims,
      "recent_claims": recent_claims,
      "notifications": notifications,
    }
  )
)
```

5.3 Testing Approach

Testing was performed to check whether the system functions correctly and meets the requirements defined during the analysis stage.

The main purpose of testing was to identify errors in individual modules as well as problems that could occur when different modules work together.

The project follows three main levels of testing:

1. Unit Testing
2. Integration Testing
3. System Testing

5.3.1 Unit Testing

Unit testing was performed on individual functions and modules of the application.

Each module was tested separately to check whether it performs its expected operation.

Examples include:

- Testing patient registration.
- Testing patient information update.
- Testing medical record creation.
- Testing automatic bill calculation.
- Testing bill number generation.
- Testing insurance claim creation.
- Testing claim amount validation.
- Testing login and logout.
- Testing form validation.
- Testing document upload.

The purpose of unit testing was to identify errors at an early stage before integrating the modules.

Sample Unit Test Cases

Test Case	Expected Result
Add valid patient	Patient should be registered successfully
Enter invalid phone number	Validation message should be displayed
Create medical record	Record should be saved successfully
Create bill	Total amount should be calculated
Enter claim greater than bill	Claim should be rejected
Valid login	User should enter the correct dashboard
Invalid login	Error message should be displayed
Update patient profile photo	Photo should be updated and displayed correctly
Delete uploaded document	Document should be removed from storage and listing
Search medical record by doctor name	Matching medical records should be returned
Logout after session	User should be redirected to the login page

5.3.2 Integrated Testing

Integration testing was performed after individual modules were tested.

The purpose was to check whether different modules communicate and work correctly with each other.

Important integrations tested include:

Patient → Medical Record

A medical record should be connected to the correct patient.

Patient → Billing

Bills should be generated for the correct patient.

Patient → Insurance Claim

Insurance claims should be connected to the correct patient.

Medical Record → Insurance Claim

The claim should use the medical record belonging to the selected patient.

Billing → Insurance Claim

The claim amount should be checked against the selected bill.

User → Role-Based Access

Users should only be able to access the pages and functions allowed for their role.

5.3.3 System/Beta Testing

System testing was performed on the complete application after integrating all major modules.

The complete system was tested from the user's point of view to check whether the main workflow works correctly.

The following activities were tested:

- User login and logout.
- Patient registration and management.
- Doctor medical record management.
- Hospital billing.
- PDF bill generation.
- Insurance claim submission.
- Claim verification.
- Claim approval and rejection.
- Document uploading.
- Dashboard information.
- Role-based access.
- Search and filtering.
- Form validation.

The system was also checked for common errors such as incorrect input, missing information, unauthorized access, and invalid claim amounts.

Sample System Test Case

Scenario: Insurance Claim Processing

1. Patient logs into the system.
2. Patient selects a medical record.
3. Patient selects the related bill.
4. Patient enters policy and claim details.
5. Patient submits the claim.
6. Insurance company views the claim.

7. Insurance company checks the submitted information.
8. Claim is approved or rejected.
9. Patient can view the updated claim status.

Expected Result:

The complete claim process should work correctly from submission to final status without data mismatch.

5.4 Modifications and Improvements

During development and testing, some changes were made to improve the functionality and usability of the system.

The major modifications and improvements include:

- Improved patient registration and profile management.
- Added patient profile photo support.
- Added search and ordering for patient and medical records.
- Added automatic patient, bill, and claim IDs.
- Added automatic calculation of total bill amount.
- Added PDF bill generation.
- Added insurance claim status management.
- Added claim amount validation.
- Added supporting document upload.
- Added role-based access for different users.
- Added company-specific access for insurance claims.
- Improved form validation and error messages.
- Added dashboards for monitoring important information.
- Added soft deletion for selected records where required.

These improvements were made based on testing results and the requirements of the project.

Overall, the modifications helped make the application more organized, easier to use, and more suitable for managing patient, billing, medical, and insurance information.

5.5 Test Cases

Test cases are prepared to check whether the different modules of the Secure Patient Data & Insurance Management System work correctly. Each test case checks a particular function by providing input and verifying the expected result.

Test Case ID	Module	Test Case	Expected Result	Status
TC01	Login	Enter valid username and password	User is successfully logged in	Pass
TC02	Login	Enter incorrect password	Error message is displayed	Pass
TC03	Patient	Add a new patient with valid details	Patient is added successfully	Pass
TC04	Patient	Enter an invalid phone number	Validation message is displayed	Pass
TC05	Patient	Search using Patient ID	Correct patient record is displayed	Pass
TC06	Medical Record	Add medical record for a patient	Record is saved successfully	Pass
TC07	Medical Record	Upload medical report	Document is uploaded and available for viewing	Pass
TC08	Billing	Create a bill with different charges	Total amount is calculated correctly	Pass
TC09	Billing	Download generated bill	Bill is downloaded as PDF	Pass
TC10	Insurance Claim	Submit a valid insurance claim	Claim is submitted successfully	Pass
TC11	Insurance Claim	Enter claim amount greater than bill amount	Validation error is displayed	Pass
TC12	Insurance	Insurance company views a claim	Patient, medical and billing details are displayed	Pass
TC13	Insurance	Change claim status to Approved	Claim status is updated successfully	Pass

TC14	Insurance	Download supporting document	Document is downloaded successfully	Pass
TC15	Access Control	Patient tries to access restricted staff page	Access is denied	Pass
TC16	Logout	Click logout	User session is ended and login page is displayed	Pass

Test Result

The above test cases were performed to verify the main functions of the system. The modules such as patient management, medical records, billing, insurance claims, authentication, document handling, and access control worked as expected. Validation and permission checks also helped prevent incorrect data and unauthorized access.

Note: In your final report, if your college expects actual testing evidence, you can also add screenshots of successful test outputs below this section.

CHAPTER 6: RESULTS AND DISCUSSION

6.1 Test Reports

After completing the implementation of the Secure Patient Data & Insurance Management System, the developed modules were tested to check whether the system works correctly and meets the required objectives.

Testing was carried out on important functions such as patient management, medical records, billing, insurance claims, authentication, and document handling. Different inputs were provided to check both valid and invalid conditions.

6.1.1 Patient Management Testing

The patient module was tested for adding, viewing, searching, and updating patient information. Required fields and validation rules were checked to ensure that incorrect information was not accepted.

Result: Patient records were successfully created, displayed, searched, and updated.

6.1.2 Medical Record Testing

The medical record module was tested by adding diagnosis, treatment, prescription, doctor, department, and visit details. The system was also checked to ensure that medical records were linked to the correct patient.

Result: Medical records were successfully stored and retrieved for the selected patients.

6.1.3 Billing Testing

The billing module was tested by entering consultation, medicine, laboratory, and other charges. The system automatically calculated the total bill amount and generated a unique bill number.

Result: Bills were generated correctly and the calculated total amount was verified.

6.1.4 Insurance Claim Testing

Insurance claim submission was tested using valid and invalid claim details. The system checked the selected patient, medical record, bill, insurance company, claim amount, and supporting documents.

The claim status was also tested for different stages such as:

- Pending
- Under Review
- Approved

- Rejected

Result: Claims were successfully submitted and processed according to their status.

6.1.5 Authentication and Access Testing

Login functionality was tested for different types of users. Access to different modules was checked according to the user's role and permissions.

Result: Authorized users were able to access the required modules, while restricted areas were not available to unauthorized users.

6.1.6 Document and PDF Testing

The system was tested for uploading supporting documents and generating digital bills in PDF format.

Result: Documents were uploaded successfully and PDF bills were generated correctly.

6.1.7 Overall Testing Result

The major functions of the system were tested successfully. The testing process helped identify and correct errors related to validation, access control, data handling, and module integration.

The final system provides the expected functionality for managing patient records, medical information, billing, and insurance claims.

6.2 User Documentation

The Secure Patient Data & Insurance Management System provides a simple web interface through which different users can perform their required activities. Access to the system depends on the user's role.

6.2.1 Login

Users first access the login page and enter their username and password. After successful authentication, the user is redirected to the appropriate dashboard.

6.2.2 Patient Module

Patients can access their profile and view relevant healthcare information. They can check their medical records, bills, and insurance claim details.

Main functions include:

- View profile
- Update profile information
- View medical records
- View bills
- Download bills
- Submit insurance claims
- Check claim status

6.2.3 Doctor Module

Doctors can access the doctor dashboard and manage patient medical information.

Main functions include:

- View patient information
- Add medical records
- Update diagnosis and treatment details
- Add prescriptions
- View previous medical records

6.2.4 Hospital/Admin Module

Hospital staff or administrators can manage the main healthcare operations.

Main functions include:

- Add and manage patients
- Search patient records
- Manage medical records
- Create and manage bills
- Generate PDF bills
- Manage uploaded documents

6.2.5 Insurance Module

Insurance companies can access claims related to their organization and review the submitted information.

Main functions include:

- View submitted claims
- Check patient and medical information
- Review bill details
- Verify supporting documents
- Update claim status
- Approve or reject claims

6.2.6 Dashboard

The dashboard provides a quick overview of system activities. Depending on the user's role, it can display information such as patients, medical records, bills, and insurance claims.

For insurance users, the dashboard can show claim counts such as:

- Pending claims
- Under Review claims
- Approved claims
- Rejected claims

6.2.7 General User Instructions

Users should follow these basic steps while using the system:

1. Open the application in a web browser.
2. Login using valid credentials.
3. Open the required module from the dashboard.
4. Enter or view the required information.
5. Save changes after entering data.
6. Logout after completing the work.

6.2.8 User Interface Screen Layouts

The following screenshots can be included in this section to explain how the system is used:

- Login Page
- Admin/Hospital Dashboard
- Patient List
- Add Patient Page
- Patient Profile
- Medical Record Page
- Billing Page
- Generated PDF Bill
- Insurance Claim Submission Page
- Insurance Claim List
- Claim Details Page
- Insurance Dashboard
- Doctor Dashboard

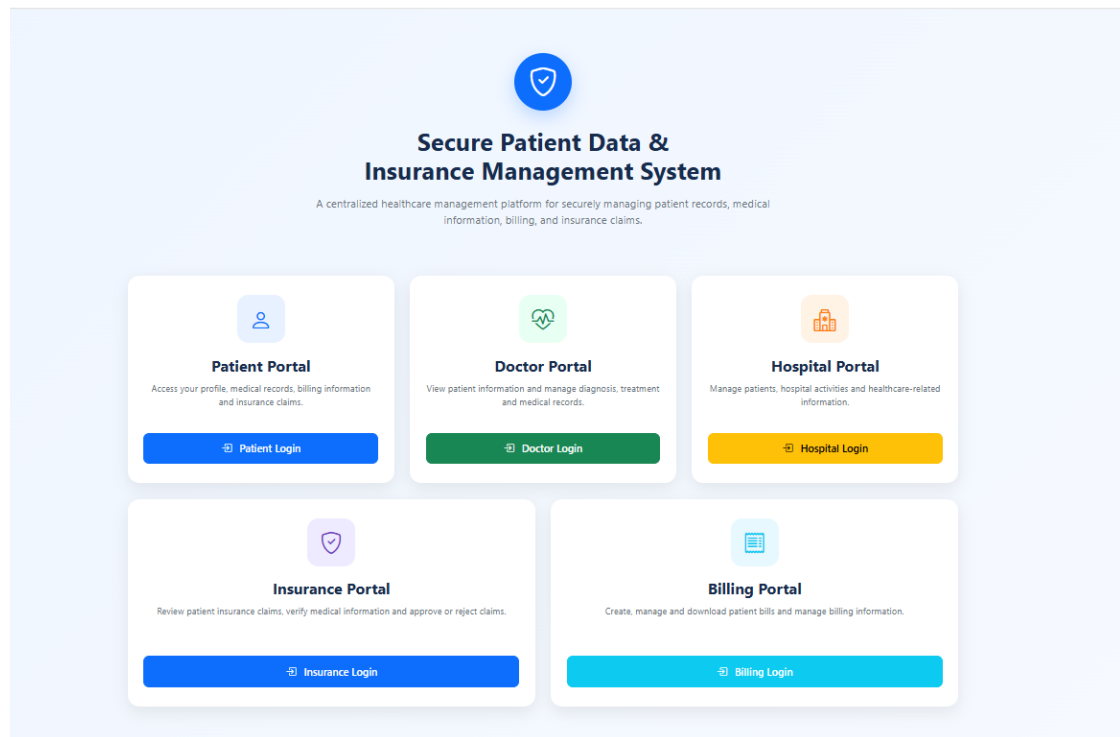


Figure 6.1: Login Page

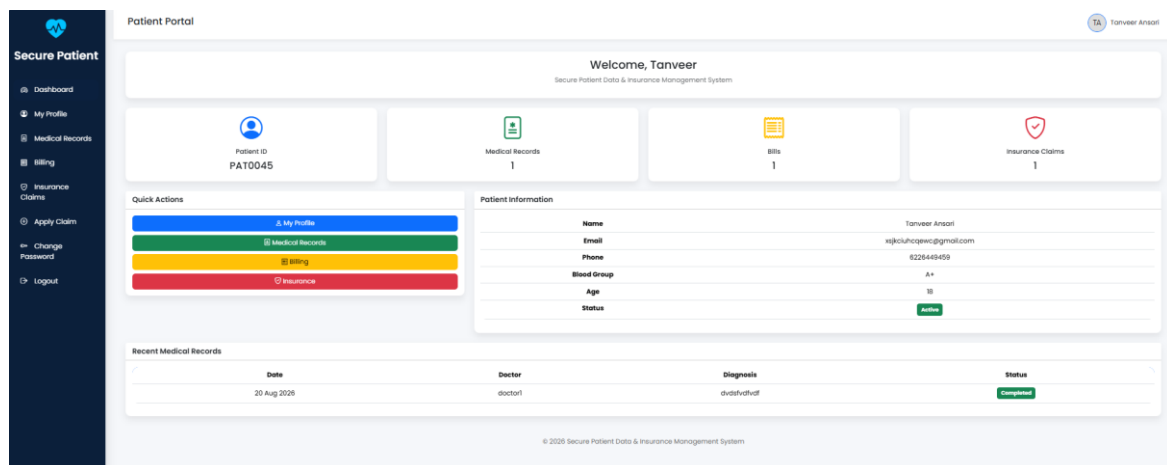


Figure 6.2: Patient Dashboard

Patient Portal

T Tanveer Ansari



Tanveer Ansari
Patient ID: PAT0045

Active

Personal Information

Patient ID
PAT0045

Email
xsjkcuhcqewc@gmail.com

Gender
Female

Blood Group
A+

Full Name
Tanveer Ansari

Phone Number
6226449459

Date of Birth
01 Jun 2008

Address
sjbcwe

Figure 6.3: Patient Profile Page

Medical Records

View and manage patient medical records.

Patient Medical Records				
Patient	Diagnosis	Treatment	Date	Action
PAT0048 - Akifah Ansari	dvedvec	dvqcerverv	23 Aug 2026	View
PAT0047 - Krishna Kumar	sdvecdcevc	fvergerg	22 Aug 2026	View
PAT0046 - pawan kumar	fdsgreg	hjr6twurthjtrdjswttaemjyululyn	21 Aug 2026	View
PAT0045 - Tanveer Ansari	dvdsvdfvdf	vdsfv	20 Aug 2026	View
PAT0044 - Muskan Shaikh	scascasc	dcscsxdawd	19 Aug 2026	View
PAT0043 - harshit vimal	tehrhrfbggfh	rththgrty	19 Aug 2026	View
PAT0042 - Rajan Kaushal	sdgdfvb	fgrfg	19 Aug 2026	View
PAT0041 - Kishan Gupta	fgnrtb	rthrth	19 Aug 2026	View
PAT0038 - Kamran Ali	sdfevf	fwwe	16 Aug 2026	View
PAT0040 - Himanshu Gupta	gdghe	dfbdggdsrgh	17 Aug 2026	View

Figure 6.4: Medical Record Page

Secure Patient System

billing

Logout

Billing Management

None

Search

Reset

New Bill

Bill No	Patient ID	Patient Name	Total	Status	Actions
BILL0020	PAT0048	Akifah Ansari	₹ 18500.00	Paid	Edit Delete Download Bill
BILL0019	PAT0047	Krishna Kumar	₹ 13500.00	Pending	Edit Delete Download Bill
BILL0016	PAT0044	Muskan Shaikh	₹ 276500.00	Paid	Edit Delete Download Bill
BILL0018	PAT0046	pawan kumar	₹ 152200.00	Paid	Edit Delete Download Bill
BILL0015	PAT0043	harshit vimal	₹ 6000.00	Pending	Edit Delete Download Bill
BILL0017	PAT0045	Tanveer Ansari	₹ 150.00	Paid	Edit Delete Download Bill
BILL0014	PAT0042	Rajan Kaushal	₹ 7000.00	Pending	Edit Delete Download Bill
BILL0013	PAT0041	Kishan Gupta	₹ 19500.00	Pending	Edit Delete Download Bill
BILL0011	PAT0026	Deepak kumar	₹ 20450.00	Pending	Edit Delete Download Bill
BILL0010	PAT0038	Kamran Ali	₹ 29305.00	Pending	Edit Delete Download Bill

Figure 6.5: Billing Page

75



SECURE PATIENT SYSTEM

Secure Patient Data & Insurance Management System

MEDICAL BILL

BILL NUMBER

BILL DATE

BILL0017

2026-08-20

PATIENT INFORMATION

Patient ID: PAT0045

Phone: 6226449459

Patient Name: Tanveer Ansari

MEDICAL RECORDS

DEPARTMENT	DIAGNOSIS / SYMPTOMS	VISIT DATE
MBBS	Diagnosis: dvdsfvdvdf; Symptoms: dflvs	2026-08-20

BILLING DETAILS

DESCRIPTION	AMOUNT
Consultation Fee	Rs. 50.00
Medicine Fee	Rs. 100.00
Laboratory Fee	Rs. 0.00
Other Charges	Rs. 0.00
TOTAL AMOUNT	Rs. 150.00

PAYMENT INFORMATION

Payment Method: Cash

Payment Status: Paid

This is a computer-generated medical bill.

Secure Patient Data & Insurance Management System

Figure 6.6: Generated PDF Bill

Patient Portal

TA Tanveer Ansari



Insurance Claims

Track all your submitted insurance claims.

Apply New Claim

Total Claims

Pending

Approved

Rejected

1

0

1

0

Search Claim ID or Insurance Company

Claim History

Claim ID	Company	Policy	Claim Amount	Approved	Status
CLM0011	ICICI Lombard	vdsv	₹ 100.00	₹ 50.00	Approved

Figure 6.7: Insurance Claim Submission

Insurance Claim Review



Patient Information

Patient ID: PAT0045

Name: Tanveer Ansari

Gender: Female

Blood Group: A+

Date of Birth: June 1, 2008

Age: 18

Phone: 6226449459

Email: xjclcluhcpevc@gmail.com

Address: sjbcwe

Medical Records

Medical Record 1 Aug 20, 2026

Doctor: Shakeri Shahn

Department: MBBS

Diagnosis: dvidshofvd

Symptoms: dhvd

Treatment: vdsfv

Prescriptions: dhvsvd

Next Visit: Aug 21, 2026

Remarks: sjdcdc

[Download Medical Report](#)

Billing Details

Bill 1	
Bill Number	BILL0017
Bill Date	Aug 20, 2026
Consultation Fee	₹ 50.00
Medicine Fee	₹ 100.00
Laboratory Fee	₹ 0.00
Other Charges	₹ 0.00
Total Amount	₹ 150.00
Payment Status	Paid
Payment Method	Cash
Download Bill PDF	

Figure 6.8: Insurance Claim Details

Insurance Dashboard

1

Pending Claims

0

Under Review

1

Approved

0

Rejected

Recent Insurance Claims

View All

Claim ID	Patient	Status	Date	Action
CLM0011	PAT0045 Tanveer Ansari	Approved	20 Aug 2026	View
CLM0007	PAT0041 Kishan Gupta	Pending	19 Aug 2026	View

Latest Notifications

CLM0007

Patient: PAT0041

A new insurance claim has been submitted for review.

19 Aug 2026, 03:36 PM

[Review Claim](#)

Quick Statistics

Total Claims	2
Pending	1
Under Review	0
Approved	1
Rejected	0

Insurance Workflow

Patient

↓

Claim Submitted

↓

Insurance Review

↓

Approve / Reject

↓

Patient Portal Updated

Figure 6.9: Insurance Dashboard

77

CHAPTER 7: CONCLUSION

7.1 Conclusion

The Secure Patient Data & Insurance Management System was developed to provide a simple and organized way of managing patient information, medical records, hospital billing, and insurance claims.

The system brings different healthcare activities together on a single web-based platform. Patients can access their personal information, medical records, bills, and insurance claims. Doctors and hospital staff can manage patient and medical information, while insurance companies can review and process claims related to their organization.

The use of Django, Python, and PostgreSQL helps the system manage data efficiently and maintain proper relationships between different records. Role-based access also ensures that users can access only the features relevant to them.

Overall, the project reduces manual paperwork, improves record management, makes information easier to retrieve, and provides a more organized process for handling insurance claims. The system also provides a good base for adding more healthcare-related features in the future.

7.1.1 Significance of the System

The Secure Patient Data & Insurance Management System is significant because it provides a single platform for managing important healthcare information. It reduces dependence on manual paperwork and makes patient, medical, billing, and insurance records easier to manage.

The system is useful because:

- It provides centralized patient information in one place.
- It makes medical records and prescriptions easier to store and access.
- It simplifies hospital billing and PDF bill generation.
- It makes the insurance claim process more organized.
- It provides role-based access so users can access only the information relevant to their role.
- It supports document upload and download for medical and insurance records.
- It reduces duplicate data, manual errors, and unnecessary paperwork.
- It improves communication between patients, doctors, hospital staff, and insurance companies.
- It provides a more organized, secure, and efficient way of handling healthcare-related information.

7.2 Limitations of the System

Although the system provides the main required healthcare and insurance functionalities, some limitations remain:

- The system is currently designed as a web application and does not have a dedicated mobile application.
- Online appointment booking is not included in the current version.
- SMS and email notifications are not implemented.
- The system does not directly connect with external insurance company systems.
- Advanced data analytics and AI-based features are not included.
- The system depends on proper internet and server availability.
- Some administrative tasks may still require manual verification.
- Advanced cloud-based backup and recovery features can be added in future versions.

These limitations do not affect the main working of the project but provide opportunities for further improvement.

7.3 Future Scope of the Project

The system can be improved further by adding new features and technologies. Some possible future enhancements are:

- **Mobile Application:** Develop Android and iOS applications for easier access.
- **Online Appointment Booking:** Allow patients to book appointments with doctors.
- **Notifications:** Add email and SMS notifications for appointments, bills, and claim updates.
- **Cloud Integration:** Store medical documents and records securely using cloud services.
- **Advanced Reports:** Provide detailed reports and graphical analysis of patients, bills, and insurance claims.
- **AI-Based Features:** Add AI-based assistance for medical information and healthcare analysis.
- **Online Payment:** Allow patients to make hospital bill payments through the system.
- **Insurance API Integration:** Connect directly with insurance company systems for faster claim processing.
- **Improved Document Management:** Provide better storage, searching, and verification of medical and insurance documents.
- **Enhanced Security:** Add stronger security measures such as two-factor authentication and improved access monitoring.

REFERENCES

The following resources and documentation were referred to during the design, development, and documentation of the Secure Patient Data & Insurance Management System:

- Official Django Documentation – <https://docs.djangoproject.com>
- Django REST Framework Documentation – <https://www.django-rest-framework.org>
- PostgreSQL Official Documentation – <https://www.postgresql.org/docs>
- Python Official Documentation – <https://docs.python.org>
- Bootstrap Documentation – <https://getbootstrap.com/docs>
- Mozilla Developer Network (MDN Web Docs) for HTML, CSS, and JavaScript references – <https://developer.mozilla.org>
- GeeksforGeeks and W3Schools tutorials for Python, Django, and web development concepts.
- Reference textbooks on software engineering and database management systems used during the course curriculum.
- Class notes, lecture material, and project guidance provided by the Department of Information Technology.

GLOSSARY

The following key terms are used throughout this report:

- API – A set of rules that allows different software applications to communicate with each other.
- CRUD – Create, Read, Update, and Delete; the four basic operations performed on stored data.
- ORM – Object-Relational Mapping; a technique used by Django to interact with the database using Python objects instead of raw SQL queries.
- Role-Based Access Control (RBAC) – A security approach in which system access is granted to users based on their assigned role, such as patient, doctor, or insurance company.
- REST – Representational State Transfer; an architectural style used for designing networked APIs.
- PDF – Portable Document Format; used in this project to generate downloadable digital bills.
- Foreign Key – A field in a database table that creates a link to the primary key of another table, used to represent relationships between records.
- Migration – A Django mechanism used to apply changes made to models as corresponding changes in the database schema.
- Dashboard – A summary screen that displays key information relevant to a particular user role.
- WSGI – Web Server Gateway Interface; a standard interface between web servers and Python web applications such as Django.